

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
CAMPUS CAMPINA GRANDE
COORDENAÇÃO DO CURSO SUPERIOR BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

LUIZ OLIVEIRA DE SOUZA NETO

JIGA DE TESTES PARA SISTEMAS EMBARCADOS

CAMPINA GRANDE – PB
2026

LUIZ OLIVEIRA DE SOUZA NETO

JIGA DE TESTES PARA SISTEMAS EMBARCADOS

Monografia apresentada ao Curso de Bacharelado em Engenharia de Computação, do Instituto Federal de Educação, Ciência e Tecnologia da Paraíba – Campus Campina Grande, em cumprimento às exigências parciais para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador: Prof. Alexandre Sales Vasconcelos

CAMPINA GRANDE – PB
2026

LUIZ OLIVEIRA DE SOUZA NETO

JIGA DE TESTES PARA SISTEMAS EMBARCADOS

Monografia apresentada ao Curso de Bacharelado em Engenharia de Computação, do Instituto Federal de Educação, Ciência e Tecnologia da Paraíba – Campus Campina Grande, em cumprimento às exigências parciais para a obtenção do título de Bacharel em Engenharia da Computação.

Aprovada em _____/_____/_____.

BANCA EXAMINADORA:

Prof. Alexandre Sales Vasconcelos(Orientador)
Instituto Federal da Paraíba (IFPB)

A definir
Instituto Federal da Paraíba (IFPB)

A definir
Instituto Federal da Paraíba (IFPB)

CAMPINA GRANDE – PB
2026

AGRADECIMENTOS

Ao orientador, Alexandre Sales Vasconcelos, pela orientação ao longo do projeto. Aos professores e colegas do curso, pelo apoio durante o desenvolvimento deste trabalho.

RESUMO

A validação de *firmware* para sistemas embarcados costuma depender de sensores reais conectados ao *hardware* final, o que limita a reprodutibilidade, a cobertura de cenários de teste e eleva o custo e o risco de dano a componentes. Soluções comerciais de *Hardware-in-the-Loop* (HIL) resolvem esse problema, mas têm custo proibitivo para laboratórios acadêmicos e pequenas empresas. Este trabalho apresenta o desenvolvimento de uma jiga de testes de baixo custo, baseada no microcontrolador STM32F7, capaz de simular sinais analógicos (DAC), digitais (GPIO/PWM) e de barramento (I2C, SPI e UART) e de adquirir as respostas de microcontroladores-alvo genéricos, com automação de rotinas de teste por meio de uma aplicação *desktop*. A solução é organizada em três camadas – controle e automação, simulação e interface física – comunicando-se por um protocolo de comandos sobre USB-CDC. Os testes são descritos em arquivos YAML e cada caso é executado na própria jiga como uma rotina, com os instantes das medições registrados com resolução de 1 µs, e o computador emite o veredito de cada caso. A jiga também emula sensores comerciais completos (MPU-6500 e BME280, no I2C e no SPI), lidos pelo DUT com os seus *drivers* normais, e varia estímulos e grandezas no tempo com formas de onda (rampa, senoide, triangular, quadrada, exponencial e outras), atualizadas por interrupção ou, no DAC, por DMA. Os resultados mostraram os conversores dentro do critério de 1% do fundo de escala na faixa útil, um custo de cerca de 28 µs por passo de rotina e, em 11 000 comandos, tempo de ida e volta máximo de 3,1 ms, sem erros nem retransmissões. A validação *black-box* é realizada com o mesmo *firmware* de comportamento conhecido em dois microcontroladores-alvo distintos, o ESP32-S3 e o Arduino Uno.

Palavras-chave: Sistemas embarcados. *Hardware-in-the-loop*. Jiga de testes. STM32. Validação de *firmware*.

ABSTRACT

Validating firmware for embedded systems usually depends on real sensors connected to the final hardware, which limits reproducibility and test coverage while increasing cost and the risk of damaging components. Commercial Hardware-in-the-Loop (HIL) solutions address this problem, but their cost is prohibitive for academic laboratories and small companies. This work presents the development of a low-cost test jig, based on the STM32F7 microcontroller, capable of simulating analog (DAC), digital (GPIO/PWM) and bus (I2C, SPI and UART) signals and of acquiring the responses of generic target microcontrollers, with test-routine automation provided by a desktop application. The solution is organized into three layers – control and automation, simulation, and physical interface – communicating through a command protocol over USB-CDC. Tests are described in YAML files, each test case is executed on the jig itself as a routine, with measurement instants recorded at 1 µs resolution, and the computer issues the verdict of each case. The jig also emulates complete commercial sensors (MPU-6500 and BME280, over I2C and SPI), read by the DUT with its regular drivers, and varies stimuli and sensor quantities over time with waveforms (ramp, sine, triangle, square, exponential and others), updated by interrupt or, on the DAC, by DMA. The results showed the converters within the 1% full-scale criterion over the usable range, an overhead of about 28 µs per routine step and, over 11,000 commands, a maximum round-trip time of 3.1 ms, with no errors or retransmissions. Black-box validation is performed with the same known-behavior firmware on two distinct target microcontrollers, the ESP32-S3 and the Arduino Uno.

Keywords: Embedded systems. Hardware-in-the-loop. Test jig. STM32. Firmware validation.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura em três camadas da jiga de testes. 24
Figura 2 – Esquema de ligação entre a jiga e o DUT. 29
Figura 3 – Arquitetura em camadas do firmware. 31
Figura 4 – Sequência de processamento de um comando direto no firmware. . 34
Figura 5 – Máquina de estados do receptor de quadros. 35
Figura 6 – Estados de execução de uma rotina na `TestExecTask`. 36
Figura 7 – Ciclo de execução de uma rotina entre o computador e a jiga. 41
Figura 8 – Diagrama de classes da aplicação *desktop*. 44
Figura 9 – Exemplo de bateria de testes. 45
Figura 10 – Fluxograma de execução de uma bateria de testes. 47
Figura 11 – Erro da leitura do ADC no *loopback* DAC→ADC (média e faixa de 16 leituras por ponto). 57
Figura 12 – Distribuição do tempo de ida e volta dos comandos (intervalos de 0,1 ms). 60
Figura 13 – Formato do quadro do protocolo PC–Jiga. 70
Figura 14 – Retransmissão após perda da resposta. 77

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais (RF). 25
Quadro 2 – Requisitos não funcionais (RNF). 26
Quadro 3 – Mapeamento de pinos dos conectores CN7 e CN8. 26
Quadro 4 – Mapeamento de pinos dos conectores CN9 e CN10. 27
Quadro 5 – Tarefas do FreeRTOS. 32
Quadro 6 – Itens de uma bateria de testes. 46
Quadro 7 – Rastreabilidade entre requisitos e níveis de verificação. 49
Quadro 8 – Critérios de aceitação das grandezas medidas. 50
Quadro 9 – Casos de teste de unidade e de integração com comandos diretos. 51
Quadro 10 – Casos de teste de rotinas, de sistema, de aceitação e de desempenho. 52
Quadro 11 – Testes automatizados e resultados na versão 0.14.0. 55
Quadro 12 – Atendimento aos requisitos. 62
Quadro 13 – Campos do quadro do protocolo. 70
Quadro 14 – Códigos de erro do NACK. 71
Quadro 15 – Comandos diretos de sistema, DAC, GPIO, PWM e ADC. 72
Quadro 16 – Comandos diretos de I2C, SPI e UART. 73
Quadro 17 – Comandos de rotina. 74
Quadro 18 – Passos admitidos em uma rotina. 75
Quadro 19 – Estados de execução de uma rotina. 77
Quadro 20 – Exemplos de quadros de comandos diretos (bytes em hexadecimal). 79
Quadro 21 – Exemplos de quadros de uma rotina (bytes em hexadecimal). . . . 79

LISTA DE TABELAS

Tabela 1 – Lista de materiais da jiga de testes. 30
Tabela 2 – Tensão de saída do DAC medida com multímetro, sem carga. 56
Tabela 3 – Resumo do *loopback* DAC→ADC na faixa útil (33 pontos, 16 leituras por ponto). 56
Tabela 4 – PWM medido com osciloscópio (CT12). 57
Tabela 5 – Tempo de ida e volta dos comandos (CT23 e CT24). 59

LISTA DE ABREVIATURAS E SIGLAS

ACK	*Acknowledgement* (confirmação)
ADC	*Analog-to-Digital Converter* (conversor analógico-digital)
DAC	*Digital-to-Analog Converter* (conversor digital-analógico)
DUT	*Device Under Test* (dispositivo sob teste)
GPIO	*General Purpose Input/Output*
HAL	*Hardware Abstraction Layer* (camada de abstração de *hardware*)
HIL	*Hardware-in-the-Loop*
I2C	*Inter-Integrated Circuit*
MCU	*Microcontroller Unit*
PWM	*Pulse Width Modulation* (modulação por largura de pulso)
SPI	*Serial Peripheral Interface*
USB-CDC	*Universal Serial Bus – Communications Device Class*

SUMÁRIO

1	**INTRODUÇÃO**	**13**
1.1	Contextualização e Descrição do Tema	13
1.2	Conceitos Trabalhados	13
1.3	Identificação da Lacuna e Formulação do Problema	14
1.4	Justificativa	14
1.5	Requisitos da Implementação	15
1.6	Objetivos	16
1.6.1	Objetivo Geral	16
1.6.2	Objetivos Específicos	16
1.7	Escopo e Limites do Trabalho	16
1.8	Uso de Inteligência Artificial	17
1.9	Organização do Documento	17
2	**FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS**	**18**
2.1	Sistemas Embarcados e o Ciclo de Validação de Firmware	18
2.2	O Problema: Limitações da Validação com Sensores Reais	19
2.3	Fundamentos da Solução: Simulação de Sinais via *Hardware-in-the-Loop*	20
2.3.1	Soluções Comerciais de HIL	20
2.3.2	Precedentes de Simulação de Sinais em Microcontroladores	21
2.3.3	Automação de Testes e Jigas Industriais	21
2.3.4	Pluma Automated Testing Tool	21
2.4	Síntese: Fundamentos que Sustentam a Proposta	22
3	**METODOLOGIA**	**23**
3.1	Caracterização da Pesquisa	23
3.2	Visão Geral da Solução	24
3.3	Requisitos	24
3.4	Projeto de Hardware	25
3.4.1	Escolha do Microcontrolador	25
3.4.2	Mapeamento de Pinos	26
3.4.3	Compatibilidade de Níveis de Tensão	27
3.4.4	Conversores Analógicos	28
3.4.5	Esquema de Ligação com o DUT	28
3.4.6	Lista de Materiais e Custo	29
3.5	Projeto de Firmware	30

3.5.1 Arquitetura em Camadas 31
3.5.2 Tarefas e Sincronização 32
3.5.3 Processamento de um Comando 33
3.5.4 Máquina de Estados do Receptor de Quadros 34
3.5.5 Interpretador de Rotinas 35
3.5.6 Emulação de Sensores 37
3.5.7 Gerador de Ondas 38
3.6 Protocolo de Comunicação PC–Jiga 39
3.6.1 Meio de Transmissão 39
3.6.2 Modelo de Comunicação 39
3.6.3 Rotinas Executadas na Jiga 40
3.6.4 Formato do Quadro 41
3.6.5 Detecção de Erros e Retransmissão 42
3.6.6 Comandos e Papéis nos Barramentos 42
3.7 Aplicação Desktop 43
3.7.1 Escolha da Linguagem 43
3.7.2 Arquitetura da Aplicação 43
3.7.3 Descrição dos Testes 45
3.7.4 Execução de uma Bateria 47
3.8 Estratégia de Verificação 48
3.8.1 Níveis de Teste 48
3.8.2 Critérios de Aceitação 49
3.8.3 Casos de Teste 50
3.8.4 Testes de Desempenho 53

4 RESULTADOS E DISCUSSÕES 54
4.1 Solução Implementada 54
4.2 Testes de Unidade 54
4.3 Testes de Integração com Comandos Diretos 55
4.3.1 DAC e ADC 56
4.3.2 PWM 57
4.3.3 I2C, SPI e UART 58
4.4 Rotinas Executadas na Jiga 58
4.5 Testes de Sistema e de Aceitação 58
4.6 Desempenho da Comunicação 59
4.7 Emulação de Sensores e Gerador de Ondas 60
4.8 Casos Executados em Bancada 61
4.9 Custo 61
4.10 Atendimento aos Requisitos 61

5 CONSIDERAÇÕES FINAIS E SUGESTÕES PARA TRABALHOS FUTUROS 63
5.1 Sugestões para Trabalhos Futuros 64

REFERÊNCIAS 66

APÊNDICES 68

APÊNDICE A – ESPECIFICAÇÃO COMPLETA DO PROTOCOLO DE COMUNICAÇÃO 69
A.1 Princípios e Convenções 69
A.2 Formato do Quadro 70
A.3 Verificação de Integridade 70
A.4 Respostas 71
A.4.1 ACK 71
A.4.2 NACK 71
A.5 Comandos Diretos 72
A.5.1 Emulação de dispositivos I2C e SPI 72
A.5.2 Emulação de sensores 74
A.6 Comandos de Rotina 74
A.6.1 Formato da Rotina 74
A.6.2 Passo ONDA 76
A.6.3 Formato dos Resultados 76
A.6.4 Estados de Execução 76
A.6.5 Regras de Execução 76
A.7 Sequenciamento e Retransmissão 77
A.8 Estado Seguro 78
A.9 Exemplos de Quadros 78

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO E DESCRIÇÃO DO TEMA

Nos últimos anos, observa-se um crescimento expressivo da presença de sistemas embarcados nos mais diversos setores, impulsionado principalmente pela expansão da Internet das Coisas (IoT), pela automação industrial e pela popularização de plataformas de microcontroladores acessíveis, como ESP32, Arduino e a família STM32. Esse avanço tem transformado a forma como dispositivos eletrônicos são projetados e validados, especialmente no que diz respeito ao ciclo de desenvolvimento e teste de *firmware*.

O desenvolvimento de *firmware* consiste numa série de etapas em que o programador escreve códigos que devem ser compilados, gravados no microcontrolador-alvo e testados quanto à sua capacidade de responder a estimulações externas. Nesse cenário, destaca-se a validação de *firmware*, que consiste em confirmar que o comportamento do dispositivo embarcado corresponde ao esperado diante de diferentes estímulos de entrada. No fluxo de desenvolvimento de projetos acadêmicos, de pesquisa e de pequenas empresas de tecnologia, essa validação é realizada, em geral, diretamente com sensores reais conectados ao *hardware* final. Diante desse contexto, torna-se relevante investigar alternativas que tornem esse processo de validação mais rápido, seguro e reprodutível, sem depender exclusivamente de sensores físicos.

1.2 CONCEITOS TRABALHADOS

Para a compreensão do problema e da solução propostos neste trabalho, é necessário situar três conceitos centrais.

Um **sistema embarcado** é um sistema computacional dedicado, composto por *hardware* e *software* projetados para executar uma função específica dentro de um dispositivo maior, geralmente sob restrições de custo, consumo de energia e tempo de resposta (Portal Embarcados, 2020a). É sobre esse tipo de sistema – e, mais especificamente, sobre o *firmware* que o controla – que a jiga de testes proposta atua.

Entende-se por ***Hardware-in-the-Loop*** **(HIL)** a técnica de simulação em tempo real na qual um *hardware* real interage com um ambiente simulado que reproduz o comportamento de sensores, atuadores ou da planta física à qual o sistema seria conectado, permitindo validar o comportamento do dispositivo sob condições controladas e repetíveis antes de sua implantação definitiva (dSPACE, s.d.). Tal conceito está diretamente relacionado à proposta deste trabalho, uma vez que a jiga desenvolvida aplica uma variação de baixo custo dessa técnica.

Por fim, adota-se para este trabalho a definição de **teste *black-box*** como

a abordagem na qual o dispositivo sob teste (DUT – *Device Under Test*) é validado exclusivamente por meio de estímulos aplicados às suas entradas e da observação das respostas em suas saídas, sem acesso ou conhecimento da estrutura interna do *firmware* avaliado (Portal Embarcados, 2020b).

1.3 IDENTIFICAÇÃO DA LACUNA E FORMULAÇÃO DO PROBLEMA

Soluções comerciais de HIL, como as oferecidas pela dSPACE e pela National Instruments, são extremamente eficazes e amplamente utilizadas pela indústria automobilística e aeroespacial (dSPACE, s.d.; National Instruments, s.d.). Apresentam, contudo, limitações quanto ao custo: os preços desses sistemas superam várias centenas de milhares de dólares, o que os torna inacessíveis para laboratórios universitários e pequenas empresas de tecnologia (seção 2.3). Diante da falta dessas opções, a alternativa amplamente adotada por desenvolvedores é o uso de abordagens manuais, nas quais sensores reais são ligados ao *hardware* final, as variáveis de ambiente são modificadas fisicamente e o resultado é monitorado observando-se o comportamento do dispositivo.

Esse método, embora funcional, apresenta limitações em termos de reprodutibilidade, velocidade e cobertura de cenários de teste. Testes sob condições ambientais específicas – como extremos de temperatura, pressões anômalas ou variações de intensidade luminosa – dificilmente podem ser reproduzidos em uma mesa de bancada; a validação de rotinas de tratamento de falhas exige simular sensores defeituosos ou sinais fora do padrão esperado; e a execução manual de testes de regressão a cada modificação do *firmware* é demorada, além de expor sensores reais – muitas vezes caros, frágeis ou de difícil acesso – ao risco de dano. Nesse sentido, evidencia-se uma carência de soluções acessíveis de simulação de sinais de sensores voltadas para o ciclo de desenvolvimento de *firmware* em laboratórios acadêmicos e pequenas equipes de engenharia.

Diante do exposto, esta pesquisa busca responder ao seguinte problema: como validar o *firmware* de sistemas embarcados de forma reprodutível e de baixo custo, simulando sinais de sensores e periféricos, sem depender de sensores reais ou de plataformas comerciais de *Hardware-in-the-Loop*?

1.4 JUSTIFICATIVA

Do ponto de vista acadêmico, o trabalho se justifica por ocupar o espaço entre duas linhas já consolidadas na literatura, discutidas no Capítulo 2. De um lado, as técnicas de teste *off-target* validam a lógica do *software* sem o *hardware* real (LINS *et al.*, 2016; BRANAM, 2025), mas não verificam o comportamento do *firmware* diante de estímulos elétricos. De outro, as plataformas comerciais de HIL verificam esse

comportamento, mas a um custo inviável para o meio acadêmico (dSPACE, s.d.; National Instruments, s.d.). Uma jiga de baixo custo, com protocolo de comunicação documentado e rotinas de teste reprodutíveis, permite validar o *firmware* executando no próprio *hardware*-alvo sem recorrer a nenhum dos dois extremos. Além disso, a documentação da arquitetura e do protocolo (Apêndice A) permite que a solução seja reproduzida e estendida em outros trabalhos e aproveitada em atividades de ensino de sistemas embarcados, a exemplo dos laboratórios baseados em microcontroladores descritos por Benovics *et al.* (2022).

Do ponto de vista prático, laboratórios acadêmicos e pequenas empresas de tecnologia – justamente os que não têm acesso a plataformas comerciais – são os que mais dependem da validação manual com sensores reais, sujeita às limitações descritas na seção anterior. Automatizar essas rotinas reduz o tempo gasto em testes de regressão a cada alteração do *firmware*, diminui o risco de dano a sensores e permite exercitar cenários de falha que dificilmente seriam reproduzidos em bancada, o que contribui para que produtos embarcados desenvolvidos com poucos recursos cheguem ao uso com maior confiabilidade.

Por fim, o tema reúne em um único artefato conteúdos centrais da formação em Engenharia de Computação – microcontroladores, sistemas operacionais de tempo real, protocolos de comunicação, desenvolvimento de *software* e verificação –, o que motivou sua escolha como Trabalho de Conclusão de Curso.

1.5 REQUISITOS DA IMPLEMENTAÇÃO

Para que a solução proposta seja capaz de suprir a lacuna identificada, é necessário que o sistema atenda, em linhas gerais, aos seguintes requisitos.

Do ponto de vista funcional, espera-se que a jiga seja capaz de gerar sinais analógicos, digitais e de comunicação configuráveis, emulando sensores e periféricos que se conectam ao microcontrolador-alvo; que adquira automaticamente as respostas do dispositivo sob teste; e que automatize, por meio de uma aplicação *desktop*, a execução de rotinas de teste e o veredito de cada caso (passou/falhou), dispensando a intervenção manual em cada cenário.

Quanto aos requisitos não funcionais, destaca-se a necessidade de baixo custo – de forma a viabilizar sua adoção por laboratórios acadêmicos e pequenas equipes de engenharia –, de portabilidade entre microcontroladores-alvo distintos e de confiabilidade na comunicação entre o computador e a jiga, de modo a não comprometer a validade dos resultados de teste. A especificação completa dos requisitos funcionais e não funcionais é apresentada no Capítulo 3.

1.6 OBJETIVOS

1.6.1 Objetivo Geral

Este trabalho tem como objetivo geral desenvolver uma jiga de testes de baixo custo, baseada no microcontrolador STM32F7, capaz de simular sinais de sensores e periféricos e de automatizar rotinas de teste por meio de uma aplicação *desktop*, a fim de validar o *firmware* de microcontroladores-alvo de forma reprodutível e sem depender de sensores reais ou de plataformas comerciais de HIL.

1.6.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

a) revisar as soluções acadêmicas e comerciais existentes para simulação e teste de *firmware* de sistemas embarcados, de modo a fundamentar as decisões de projeto (Capítulo 2);

b) implementar *firmware* no STM32F7 capaz de gerar sinais analógicos (DAC), digitais (GPIO) e de comunicação (I2C, SPI) configuráveis via interface serial;

c) desenvolver uma aplicação *desktop* para configuração de pinos, criação de rotinas de teste e visualização de resultados;

d) implementar mecanismo de leitura de respostas do microcontrolador-alvo para validação automática de critérios de aceitação (passou/falhou);

e) validar a solução com pelo menos dois microcontroladores-alvo distintos (ex.: ESP32 e Arduino Uno), comparando os resultados obtidos com os comportamentos previamente esperados.

1.7 ESCOPO E LIMITES DO TRABALHO

Delimitam-se, a seguir, os limites do trabalho:

- o projeto não pretende substituir plataformas industriais de HIL de alta precisão e alta velocidade;
- não serão desenvolvidos modelos matemáticos complexos de sensores; a simulação é baseada em perfis de sinais configuráveis pelo usuário;
- a interface física entre a jiga e o microcontrolador-alvo é feita por *jumpers*, sem o desenvolvimento de uma placa de interface dedicada;

- o sistema é voltado para testes *black-box* (seção 1.2), sem instrumentação interna do *firmware* do dispositivo sob teste;
- o projeto não aborda testes de desempenho de tempo real com precisão de nanossegundos, limitando-se à faixa de operação dos periféricos do STM32F7.

1.8 USO DE INTELIGÊNCIA ARTIFICIAL

O desenvolvimento deste trabalho observa princípios de integridade acadêmica e de uso responsável de ferramentas de Inteligência Artificial (IA). As ferramentas de IA foram empregadas exclusivamente como assistentes de apoio, não substituindo a autoria, o julgamento técnico ou a responsabilidade do autor sobre o conteúdo produzido: a IA Gemini foi utilizada para a revisão ortográfica e gramatical dos textos e para o auxílio na geração de tabelas, enquanto a IA Claude foi utilizada como agente de apoio à construção do código, à solução de *bugs* e à sugestão de correções na lógica do *software* e do *firmware*. Em ambos os casos, todo o conteúdo gerado com auxílio dessas ferramentas foi revisado, validado e adaptado pelo autor, que assume total responsabilidade pela originalidade, integridade e por todo o conteúdo final apresentado neste trabalho.

1.9 ORGANIZAÇÃO DO DOCUMENTO

No Capítulo 2, são apresentados os trabalhos relacionados e o estado da prática em soluções de simulação e teste de *firmware* para sistemas embarcados.

No Capítulo 3, é apresentada a metodologia adotada no desenvolvimento da jiga de testes, incluindo a arquitetura da solução, os módulos de cada camada e os requisitos do sistema.

No Capítulo 4, são apresentados e discutidos os resultados obtidos com a validação da jiga desenvolvida.

No Capítulo 5, são apresentadas as considerações finais e as sugestões para trabalhos futuros.

No Apêndice A, é apresentada a especificação completa do protocolo de comunicação PC-Jiga.

2 FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS

Este capítulo constrói o embasamento teórico necessário para sustentar a proposta apresentada no Capítulo 1. A discussão é organizada em três etapas: primeiro, situam-se os conceitos indispensáveis para compreender o ciclo de validação de *firmware* em sistemas embarcados (seção 2.1); em seguida, aprofunda-se, com apoio da literatura, o problema da dependência de sensores reais nesse ciclo (seção 2.2); por fim, apresentam-se os fundamentos que justificam a solução adotada – uma jiga de simulação de sinais baseada em microcontrolador (seção 2.3). O capítulo é encerrado com uma síntese que conecta essas três camadas à proposta deste trabalho (seção 2.4).

2.1 SISTEMAS EMBARCADOS E O CICLO DE VALIDAÇÃO DE FIRMWARE

A validação de *firmware* é uma etapa crítica do ciclo de desenvolvimento de sistemas embarcados, pois é nela que se verifica se o comportamento do dispositivo corresponde ao especificado diante de estímulos de entrada controlados. Em geral, essa validação depende diretamente das características do *hardware*-alvo: frequência de operação, tensões de alimentação, número e tipo de periféricos disponíveis (conversores analógico-digitais e digitais-analógicos, barramentos de comunicação, temporizadores) determinam quais sinais podem ser gerados e capturados durante os testes (STMicroelectronics, s.d.a). É por isso que, antes de qualquer decisão de projeto, é necessário compreender tanto os limites do microcontrolador que atuará como núcleo da jiga quanto os requisitos mínimos de comunicação com o dispositivo sob teste.

Uma prática recomendada no desenvolvimento de sistemas embarcados é projetar o *hardware* e o *firmware* já pensando em como eles serão testados – abordagem conhecida como *design for test*. Segundo a Altium Resources (2024), decisões tomadas ainda na fase de projeto, como a exposição de pontos de teste e a padronização de interfaces de comunicação, reduzem significativamente o esforço necessário para validar o dispositivo posteriormente. Essa recomendação reforça a relevância de se definir, desde o início, um protocolo de comunicação bem especificado entre a jiga e o microcontrolador-alvo (detalhado no Capítulo 3).

Quanto às estratégias de teste em si, a literatura técnica costuma classificá-las conforme o acesso que se tem à implementação interna do *firmware*. Testes *white-box* exigem conhecimento do código-fonte e frequentemente dependem de instrumentação interna, enquanto testes *black-box* validam o dispositivo exclusivamente pela observação das respostas a estímulos aplicados em suas entradas, sem qualquer acesso à lógica interna (Portal Embarcados, 2020a; Fidus Systems, 2025). Essa distinção é

relevante para este trabalho porque a jiga proposta adota a abordagem *black-box*: ela não pressupõe nenhum conhecimento prévio do *firmware* do dispositivo-alvo, o que amplia sua aplicabilidade a microcontroladores de fabricantes e arquiteturas distintas.

Definidos esses conceitos, a seção seguinte aprofunda, com base na literatura, por que a prática mais comum de validação – conectar sensores reais ao *hardware* final – se torna uma limitação para laboratórios acadêmicos e pequenas equipes de desenvolvimento.

2.2 O PROBLEMA: LIMITAÇÕES DA VALIDAÇÃO COM SENSORES REAIS

A prática mais difundida para validar *firmware* de sistemas embarcados consiste em conectar sensores e atuadores reais ao *hardware* final e observar o comportamento do dispositivo diante de estímulos físicos efetivamente aplicados (Portal Embarcados, 2020b). Embora simples de implementar, esse método impõe restrições severas de reprodutibilidade: condições ambientais específicas – variações de temperatura, luminosidade ou pressão, por exemplo – são difíceis de reproduzir de forma controlada em bancada, e a repetição de um mesmo cenário de teste em diferentes execuções raramente produz condições idênticas.

Parte da comunidade acadêmica já reconhece essa limitação e propõe alternativas parciais. Lins *et al.* (2016) investigam uma metodologia de desenvolvimento dirigido por testes para sistemas embarcados de tempo real, na qual o código é separado de suas dependências físicas por meio de substitutos estruturais (*mocks*, *stubs* e *fakes*), permitindo validar a lógica do *firmware* antes mesmo da existência do *hardware* final. Na mesma direção, Branam (2025) descreve o uso de *frameworks* como Ceedling (que combina Unity e CMock) para testes unitários *off-target*, nos quais os periféricos de *hardware* são substituídos por funções simuladas. Ambas as abordagens são valiosas, mas compartilham uma limitação em comum: validam exclusivamente a lógica do *software*, sem verificar o comportamento do *firmware* executando de fato sobre o *hardware*-alvo e reagindo a estímulos elétricos reais – justamente o tipo de validação que este trabalho pretende viabilizar de forma acessível.

Barbosa (s.d.) e Mendonça (2019) chegam a uma constatação semelhante ao definirem metodologias de teste específicas para sistemas embarcados críticos e para medidores inteligentes, respectivamente: em ambos os casos, os autores destacam a dificuldade de se estabelecer um ambiente de teste padronizado e repetível quando a validação depende de instrumentação física montada manualmente para cada cenário. Cabral (2020) reforça esse ponto ao relatar as dificuldades enfrentadas para simular condições de falha de forma controlada durante o desenvolvimento de um sistema de detecção de anomalias embarcado.

Uma alternativa mais próxima da proposta deste trabalho é discutida por Benovics *et al.* (2022), que descrevem laboratórios remotos de ensino baseados em

microcontroladores STM32 e ESP8266, capazes de gerar sinais de tensão e PWM para simular condições de teste à distância. Embora a arquitetura demonstre a viabilidade técnica de se usar um microcontrolador para gerar estímulos controlados e monitorar respostas sem sensores físicos, a proposta tem finalidade educacional: destina-se ao ensino de conceitos de eletrônica, e não à automação do ciclo de validação de *firmware* de um dispositivo sob teste.

Em conjunto, esses trabalhos evidenciam duas lacunas complementares: de um lado, as abordagens de teste *off-target* (*mocks*, TDD) resolvem a validação da lógica do *software*, mas não a do comportamento eletroeletrônico do *firmware*; de outro, as arquiteturas de simulação de sinais baseadas em microcontrolador – como a de Benovics *et al.* (2022) – comprovam a viabilidade técnica da abordagem, mas não são aplicadas à automação de rotinas de teste de *firmware*. É exatamente nessa lacuna que se insere a jiga de testes proposta neste trabalho, e é para justificá-la que a seção seguinte discute os fundamentos da solução adotada.

2.3 FUNDAMENTOS DA SOLUÇÃO: SIMULAÇÃO DE SINAIS VIA *HARDWARE-IN-THE-LOOP*

Diante da necessidade de simular sinais de sensores e periféricos de forma controlada e repetível, discutida na seção anterior, a técnica de *Hardware-in-the-Loop* (HIL) torna-se relevante por permitir que um *hardware* real interaja com um ambiente simulado, sem exigir a presença dos sensores e atuadores físicos definitivos. A subseção a seguir caracteriza as soluções comerciais que já aplicam esse princípio, para em seguida discutir por que sua adoção direta não é viável no contexto deste trabalho.

2.3.1 Soluções Comerciais de HIL

Empresas como dSPACE (dSPACE, s.d.) e National Instruments (National Instruments, s.d.) oferecem plataformas de HIL amplamente utilizadas nas indústrias automobilística e aeroespacial, capazes de simular em tempo real toda a planta física à qual um sistema embarcado seria conectado, com alta fidelidade e integração com ferramentas de modelagem como o MATLAB/Simulink. Essa robustez, no entanto, tem um custo: sistemas desse porte ultrapassam facilmente a casa das centenas de milhares de dólares, o que os torna inviáveis para laboratórios acadêmicos e pequenas equipes de desenvolvimento – exatamente o público para o qual a jiga proposta neste trabalho é pensada. O princípio de simulação em tempo real permanece válido e desejável; o que se busca aqui é uma forma de aplicá-lo a um custo compatível com esse contexto.

2.3.2 Precedentes de Simulação de Sinais em Microcontroladores

A viabilidade de reproduzir o princípio do HIL em um microcontrolador de baixo custo, em vez de uma plataforma industrial dedicada, já é indicada por trabalhos anteriores. Qu & Lin (2012) descrevem um sistema de simulação de sensores de aceleração construído sobre um microcontrolador STM32, capaz de gerar sinais que emulam a saída de um acelerômetro real para fins de teste. Esse resultado é relevante para este trabalho porque demonstra, especificamente na família STM32, a capacidade de um microcontrolador de médio custo gerar sinais de simulação com fins de validação – o mesmo princípio que a jiga aqui proposta generaliza para múltiplos tipos de sinal (analógico, digital e de barramento) e para múltiplos microcontroladores-alvo, e não apenas para um sensor específico. Somado à arquitetura de laboratório remoto STM32/ESP8266 de Benovics *et al.* (2022) discutida na seção anterior, esse precedente sustenta a escolha do STM32F7 como núcleo da camada de simulação da jiga.

2.3.3 Automação de Testes e Jigas Industriais

No que diz respeito à automação das rotinas de teste, soluções industriais como a jiga automatizada da Engenharia Híbrida (s.d.) já demonstram a integração entre gravação de *firmware*, teste funcional e rastreabilidade em uma única aplicação de *software*. Essas soluções, contudo, são projetadas para inspeção de qualidade em linhas de produção em série, com o *firmware* já finalizado – não para o ciclo iterativo de desenvolvimento, no qual o *firmware* é testado repetidamente a cada alteração de código. Do lado do *software* de automação, TheEmbeddedKit (2024) discute alternativas ao Robot Framework para orquestração de testes de *hardware* embarcado, evidenciando que a decisão de como estruturar o motor de execução de testes – detalhado no Capítulo 3 – é, por si só, um problema de projeto relevante, e não um detalhe de implementação secundário.

2.3.4 Pluma Automated Testing Tool

O Pluma é uma ferramenta de teste automatizado projetada especificamente para sistemas embarcados Linux. Oferece teste de *hardware* e *software* em um ambiente de linha de comando, com suporte a protocolos como BLE, CAN e Modbus. É uma solução relevante por também automatizar testes de *hardware* embarcado, mas não se aplica ao contexto deste trabalho: destina-se a sistemas embarcados baseados em Linux, e não à simulação de sinais de sensores em microcontroladores *bare-metal*, foco central da jiga proposta.

2.4 SÍNTESE: FUNDAMENTOS QUE SUSTENTAM A PROPOSTA

Os três níveis discutidos neste capítulo se articulam diretamente com o problema e o objetivo geral apresentados na Introdução. Os conceitos de validação de *firmware* e teste *black-box* (seção 2.1) definem o vocabulário e os limites dentro dos quais a solução deve operar. Os trabalhos que caracterizam o problema (seção 2.2) mostram que a validação de *firmware* sem sensores reais é uma lacuna reconhecida tanto por abordagens de teste *off-target* – que resolvem apenas a lógica do *software* – quanto por arquiteturas de simulação em microcontrolador de finalidade educacional, que comprovam a viabilidade técnica sem endereçar a automação do ciclo de teste. Por fim, os fundamentos da solução (seção 2.3) mostram que o princípio de simulação em tempo real do HIL é desejável, mas inacessível em sua forma comercial, e que já existem precedentes de sua aplicação em microcontroladores STM32 de baixo custo.

Diante desse cenário, este trabalho propõe uma jiga de testes que aplica o princípio do HIL comercial em uma escala compatível com laboratórios acadêmicos: um microcontrolador STM32F7 como núcleo de simulação de sinais, comunicando-se com uma aplicação *desktop* responsável pela automação das rotinas de teste. A caracterização detalhada dessa arquitetura, de seus requisitos e das decisões de projeto que a fundamentam é apresentada no Capítulo 3.

3 METODOLOGIA

Este capítulo apresenta a solução proposta para o problema formulado no Capítulo 1 e as decisões que a fundamentam. Como a seção 2.3 discutiu apenas os princípios e os precedentes da simulação de sinais em microcontroladores, e não uma arquitetura completa, o capítulo caracteriza primeiro a pesquisa (seção 3.1), descreve a solução como um todo (seção 3.2), formaliza os requisitos que ela deve atender (seção 3.3) e, em seguida, detalha o *hardware* (seção 3.4), o *firmware* (seção 3.5), o protocolo de comunicação (seção 3.6), a aplicação *desktop* (seção 3.7) e a estratégia de verificação (seção 3.8). Cada escolha de projeto é acompanhada de sua justificativa.

3.1 CARACTERIZAÇÃO DA PESQUISA

Quanto à natureza, trata-se de uma pesquisa aplicada, pois busca gerar conhecimento voltado à solução de um problema prático e específico – a validação de *firmware* sem sensores reais –, materializado em um artefato tecnológico (PRODANOV; FREITAS, 2013). Quanto à abordagem, é predominantemente quantitativa, já que o funcionamento da solução é avaliado por grandezas mensuráveis (tensões, frequências, razões cíclicas e taxa de acerto dos vereditos), comparadas a valores de referência.

Quanto aos procedimentos técnicos, combinam-se dois tipos. Na etapa de fundamentação (Capítulo 2), realiza-se pesquisa bibliográfica sobre validação de *firmware*, técnicas de teste e simulação de sinais. Na etapa de validação, adota-se o procedimento experimental: define-se o objeto de estudo, selecionam-se as variáveis que podem influenciá-lo e estabelecem-se as formas de controle e de observação de seus efeitos (GIL, 2017). Aqui, as variáveis manipuladas são os estímulos aplicados pela jiga, e os efeitos observados são as respostas do DUT e os vereditos emitidos.

O objeto de estudo é a própria jiga. Os dispositivos sob teste empregados na validação são o ESP32 e o Arduino Uno, escolhidos por serem plataformas populares em projetos acadêmicos (Capítulo 1), por operarem em níveis lógicos distintos (3,3 V e 5 V) e por pertencerem a arquiteturas diferentes (Xtensa e AVR), o que permite avaliar a portabilidade da solução (RNF02). Os dados são coletados pela própria jiga – respostas adquiridas e vereditos registrados pela aplicação (RF09) – e, nos testes de integração, confrontados com medições de instrumentos de referência, conforme a seção 3.8.

O trabalho é conduzido em cinco etapas: (i) levantamento dos requisitos a partir do problema e dos objetivos (seção 3.3); (ii) projeto do *hardware* e escolha do microcontrolador (seção 3.4); (iii) implementação do *firmware* e da aplicação *desktop* (seções 3.5 a 3.7); (iv) verificação em três níveis – unidade, integração e sistema

(seção 3.8); e (v) validação *black-box* em pelo menos dois microcontroladores-alvo, cujos dados são apresentados no Capítulo 4.

3.2 VISÃO GERAL DA SOLUÇÃO

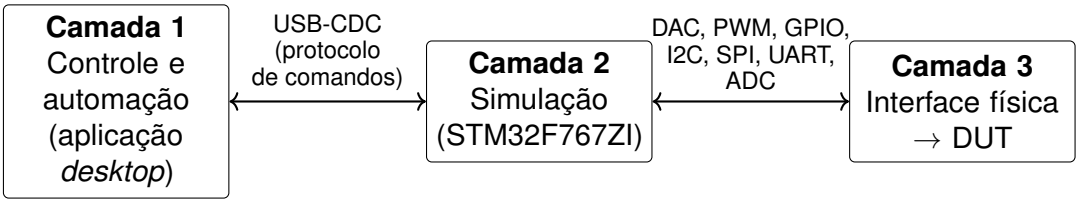
A solução proposta é uma jiga de testes composta por três camadas, conforme a Figura 1:

a) **camada de controle e automação**: uma aplicação *desktop*, executada em um computador, que define as rotinas de teste, envia comandos à jiga, recebe as respostas e emite o veredito de cada caso (passou/falhou);

b) **camada de simulação**: o *firmware* do microcontrolador STM32F767ZI, que executa os comandos e as rotinas de teste recebidos, gera os estímulos elétricos (analógicos, digitais, PWM e de barramento) e adquire as respostas do dispositivo sob teste (DUT);

c) **camada de interface física**: os conectores e *jumpers* que ligam os pinos da jiga aos pinos do DUT.

Figura 1 – Arquitetura em três camadas da jiga de testes.



Fonte: elaborado pelo autor.

A separação em camadas foi adotada por duas razões. A primeira é a portabilidade: como a jiga se relaciona com o DUT apenas por sinais elétricos, sem instrumentá-lo, ela mantém a abordagem *black-box* definida na seção 1.2 e pode ser aplicada a microcontroladores de fabricantes e arquiteturas distintas (Portal Embarcados, 2020b). A segunda é a testabilidade: com um protocolo de comandos bem definido entre as camadas 1 e 2, cada uma pode ser verificada isoladamente, o que corresponde à prática de *design for test* recomendada por Altium Resources (2024).

3.3 REQUISITOS

Os requisitos foram derivados do problema (Capítulo 1) e dos objetivos específicos. Cada requisito recebe um identificador, usado na seção 3.8 para rastrear qual teste o comprova. O Quadro 1 apresenta os requisitos funcionais e o Quadro 2, os não funcionais.

Quadro 1 – Requisitos funcionais (RF).

ID	Requisito	Critério de aceitação
RF01	Gerar tensão analógica configurável em dois canais (DAC1 e DAC2), na faixa útil de 0,1 a 3,2 V (subseção 3.4.4).	Tensão medida na saída coincide com a comandada, dentro da tolerância definida na subseção 3.8.2.
RF02	Gerar níveis lógicos em quatro saídas digitais (GPIO1–4).	Nível medido na saída coincide com o comandado.
RF03	Gerar sinal PWM em três canais, com frequência e razão cíclica configuráveis.	Frequência e razão cíclica medidas coincidem com as comandadas, dentro da tolerância definida na subseção 3.8.2.
RF04	Emular dispositivos escravos nos barramentos I2C e SPI e comunicar-se como mestre com periféricos do DUT.	Dado escrito por uma extremidade é lido íntegro na outra.
RF05	Adquirir respostas do DUT: quatro entradas digitais (GPIO5–8), três canais analógicos (ADC) e dados pela UART.	Valor adquirido coincide com o aplicado ao pino.
RF06	Receber comandos do computador por USB-CDC e confirmar cada um com ACK ou NACK.	Todo comando válido recebe ACK e todo inválido recebe NACK.
RF07	Permitir, na aplicação, configurar os pinos e criar baterias de casos de teste.	Bateria criada é salva e reexecutada sem alteração manual.
RF08	Executar rotinas automaticamente e emitir o veredito (passou/falhou) de cada caso.	Veredito coincide com o resultado esperado em casos de resposta conhecida.
RF09	Registrar e exibir os resultados de cada execução.	Relatório contém estímulos, respostas e vereditos.
RF10	Executar na própria jiga um caso de teste completo por comando (estímulos, esperas e aquisições), registrando o instante de cada medição.	Rotina executada integralmente, com instantes dentro da tolerância definida na subseção 3.8.2.
RF11	Emular sensores comerciais (MPU-6500 e BME280) no I2C e no SPI, de modo que o DUT os leia com o seu *driver* normal.	Valores lidos pelo DUT iguais aos simulados, dentro da tolerância do caso.
RF12	Variar no tempo os estímulos e as grandezas dos sensores emulados, com formas de onda definidas na bateria.	Valores nos instantes previstos pela forma, conferidos na jiga e no osciloscópio.

Fonte: elaborado pelo autor.

3.4 PROJETO DE HARDWARE

3.4.1 Escolha do Microcontrolador

O núcleo da camada de simulação é o microcontrolador STM32F767ZI, da família STM32F7 (STMicroelectronics, s.d.a), utilizado em uma placa de desenvolvimento com o formato Nucleo-144. A escolha se justifica por três motivos. Primeiro, a família STM32 já foi empregada com sucesso para gerar sinais de simulação de sensores (QU; LIN, 2012) e de estímulo em laboratórios remotos (BENOVICS *et al.*, 2022), o que reduz o risco técnico do projeto. Segundo, o dispositivo reúne em um só *chip* todos os periféricos exigidos pelos requisitos: dois canais de DAC, três conversores analógico-digitais, temporizadores com saída PWM, I2C, SPI, UART e USB, dispensando componentes externos e mantendo o baixo custo (RNF01). Terceiro, a placa de desenvolvimento já

Quadro 2 – Requisitos não funcionais (RNF).

ID	Requisito	Critério de aceitação
RNF01	Baixo custo, compatível com laboratórios acadêmicos e pequenas equipes.	Lista de materiais com custo total registrada e comparada às soluções comerciais.
RNF02	Portabilidade: operar com DUTs distintos sem modificar o *firmware* deles.	Validação bem-sucedida em pelo menos dois microcontroladores-alvo.
RNF03	Confiabilidade da comunicação PC–jiga, sem perda silenciosa de comandos.	Comandos corrompidos ou perdidos são detectados e sinalizados.
RNF04	Reprodutibilidade: a mesma rotina deve produzir os mesmos estímulos.	Execuções repetidas de uma mesma rotina produzem estímulos idênticos.
RNF05	Segurança do DUT: os níveis de tensão aplicados não devem danificá-lo.	Todos os pinos operam em 3,3 V ou com proteção documentada (subseção 3.4.3).
RNF06	Modularidade: camadas independentes e protocolo documentado.	Protocolo especificado no Apêndice A e módulos testáveis isoladamente.

Fonte: elaborado pelo autor.

integra o programador/depurador e expõe os pinos em conectores padronizados, o que dispensa o desenvolvimento de uma placa dedicada, conforme o escopo do trabalho (Capítulo 1).

3.4.2 Mapeamento de Pinos

Os sinais da jiga foram distribuídos nos conectores da placa conforme os Quadros 3 e 4. O critério de distribuição foi agrupar por função: estímulos analógicos e PWM em um conector, entradas e saídas digitais em outro, e os barramentos e as entradas analógicas em um terceiro, de modo a facilitar a ligação por *jumpers* ao DUT e reduzir erros de conexão.

Quadro 3 – Mapeamento de pinos dos conectores CN7 e CN8.

Conector	Posição	Sinal	Pino	Observação
CN7	2D	SPI2_MOSI	PB15	SPI (parte)
CN7	9D	DAC1	PA4	Saída analógica
CN7	5E	DAC2	PA5	Saída analógica
CN7	6E	PWM3/1 (TIM3_CH1)	PA6	Saída PWM
CN8	4D, 5D, 6D, 7D	3V3, 5V, GND, GND	–	Alimentação
CN8	1E–4E	GPIO1–4 (saída)	PC8, PC9, PC10, PC11	Estímulo digital
CN8	5E–8E	GPIO5–8 (entrada)	PC12, PD2, PG2, PG3	Aquisição digital; tolerantes a 5 V

Fonte: elaborado pelo autor.

No projeto do STM32CubeMX, as entradas GPIO5 a GPIO8 aparecem com os rótulos `GPIO1_IN` a `GPIO4_IN`, que numeram as entradas entre si; a numeração do protocolo e deste texto é a dos quadros acima.

Quadro 4 – Mapeamento de pinos dos conectores CN9 e CN10.

Conector	Posição	Sinal	Pino	Observação
CN9	1D, 2D, 3D	A0, A1, A2 (ADC)	PA3, PC0, PC3	Aquisição analógica
CN9	10D, 11D	I2C2_SCL, I2C2_SDA	PF1, PF0	Barramento I2C
CN9	13D, 14D	UART4_RX, UART4_TX	PD0, PD1	Comunicação serial com o DUT
CN9	1E	SPI2_CS (GPIO)	PD7	Seleção de *chip* SPI
CN10	5D	SPI2_MISO	PC2	SPI (restante)
CN10	16D	SPI2_SCK	PB10	SPI (restante)
CN10	15D	PWM2/1 (TIM2_CH1)	PA0	Saída PWM
CN10	2E	PWM1/1 (TIM1_CH1)	PE9	Saída PWM

Fonte: elaborado pelo autor.

3.4.3 Compatibilidade de Níveis de Tensão

Os pinos do STM32F767ZI operam com níveis lógicos de 3,3 V. Como a jiga deve conectar-se a DUTs de famílias distintas, a compatibilidade de níveis é o principal cuidado para atender ao RNF05. Assumem-se, neste trabalho, dois DUTs de referência: o ESP32, que opera em 3,3 V, e o Arduino Uno, que opera em 5 V. As diretrizes adotadas são:

- as saídas da jiga (GPIO1–4, PWM, DAC e SPI/I2C quando geradas por ela) operam em 3,3 V, nível compatível com o ESP32 e normalmente reconhecido como nível alto por entradas de 5 V, embora com pouca margem de ruído no Arduino Uno;
- as entradas digitais GPIO5–8 foram alocadas em pinos tolerantes a 5 V (FT), para que saídas de DUTs de 5 V possam ser adquiridas sem conversor de nível;
- as entradas analógicas (ADC) e as saídas do DAC operam na faixa de 0 a 3,3 V; sinais analógicos de 5 V não podem ser ligados diretamente às entradas do ADC e exigem divisor de tensão;
- nenhuma saída de 5 V de um DUT deve ser conectada a um pino da jiga que não seja tolerante a 5 V, nem a um DUT de 3,3 V, como o ESP32, que não tolera 5 V em seus pinos;
- em barramentos I2C entre dispositivos de tensões distintas, é necessário conversor de nível ou *pull-ups* referenciados a 3,3 V.

A tolerância dos pinos FT tem duas condições, ambas da folha de dados do microcontrolador (STMicroelectronics, s.d.b). Com a jiga alimentada (VDD = 3,3 V), a tensão máxima em um pino FT é o menor valor entre VDD + 3,6 V e 5,5 V, isto é, 5,5 V; com a jiga desligada, o limite cai para 3,6 V. Por isso, a ordem de energização é

uma restrição de uso: a jiga deve ser ligada antes do DUT de 5 V e desligada depois dele, o que se cumpre naturalmente quando os dois são alimentados pelo mesmo computador e a jiga é conectada primeiro. A segunda condição é que, acima de 4 V, os resistores internos de *pull-up* e *pull-down* do pino precisam estar desligados. As entradas GPIO5–8 são, portanto, configuradas sem resistor interno; quando o DUT exigir um resistor, como em uma saída de dreno aberto, ele é externo e escolhido conforme a tensão do DUT. Sem resistor e sem ligação, uma entrada solta lê um valor indefinido, o que é esperado e não indica defeito. No sentido inverso, as saídas de 3,3 V da jiga são reconhecidas como nível alto pelo ATmega328P do Arduino Uno, cujo limiar mínimo é de 0,6 VCC, ou 3,0 V em 5 V (Microchip Technology, s.d.), com uma margem de apenas 0,3 V, suficiente para ligações curtas em bancada.

3.4.4 Conversores Analógicos

O DAC opera com o *buffer* de saída ligado. As medições com multímetro (Capítulo 4) mostraram que, com o *buffer*, a saída é precisa no meio da escala (erro de ± 3 mV no código 2048), mas satura a cerca de 50 mV de cada trilho: o código 0 dá aproximadamente 50 mV, e o código 4095, cerca de 3,25 V. Desligar o *buffer* aproximaria a saída dos trilhos, mas deixaria o pino com impedância de saída da ordem de quilo-ohms, e a carga de uma entrada do DUT derrubaria a tensão justamente no meio da faixa, onde ela é mais usada. Manteve-se o *buffer* e definiu-se uma **faixa útil** de 0,1 a 3,2 V (códigos 124 a 3971), na qual vale o critério de 1% do fundo de escala (subseção 3.8.2); fora dela, a saturação é uma limitação conhecida e documentada. O estado seguro usa o código 0, isto é, cerca de 50 mV, valor fixo e inofensivo para o DUT, em vez de desligar o DAC, o que deixaria o pino flutuando.

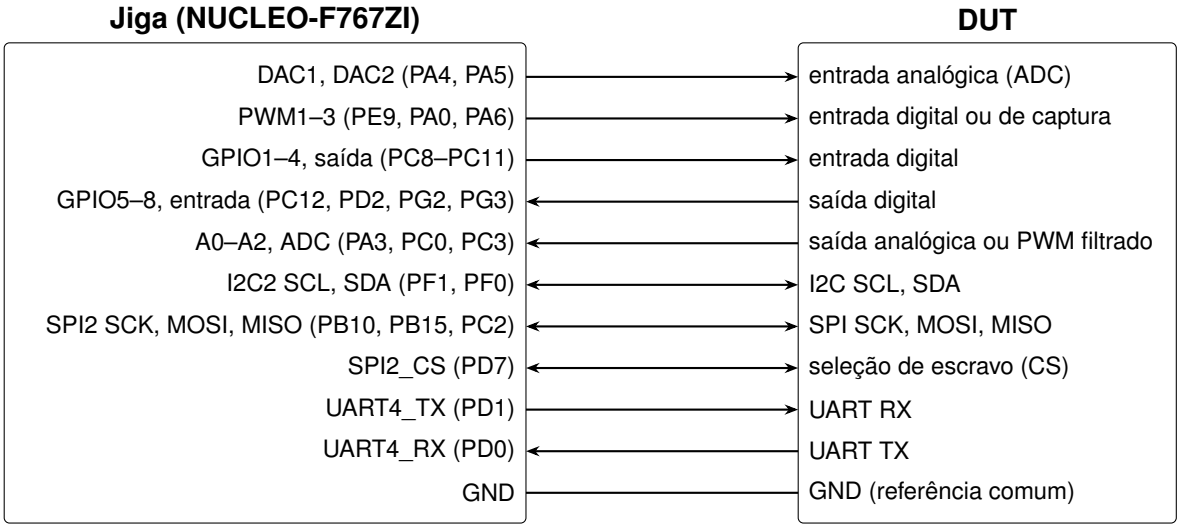
Cada canal de ADC usa um conversor próprio (ADC1, ADC2 e ADC3), com resolução de 12 bits, referência de 3,3 V e tempo de amostragem de 480 ciclos, o maior disponível: com ele, o capacitor de amostragem tem tempo de carregar mesmo quando a fonte do sinal, no DUT, tem impedância de alguns quilo-ohms. Cada ADC_READ faz uma única conversão, por varredura (*polling*), pois o tempo de conversão, de poucos microssegundos, é desprezível diante da comunicação.

3.4.5 Esquema de Ligação com o DUT

Como a interface física é feita por *jumpers*, sem placa dedicada (Capítulo 1), o esquema de ligação da Figura 2 faz o papel do esboço de placa: ele define, para cada grupo de sinais, o pino da jiga, a função correspondente no DUT e o sentido do sinal. Apenas o lado da jiga é fixo; os pinos do lado do DUT dependem do *firmware* avaliado e são indicados pelo usuário na bateria de testes.

Algumas regras de ligação decorrem do esquema. O GND da jiga e o do DUT devem ser interligados, pois todas as tensões aplicadas e medidas são referenciadas a

Figura 2 – Esquema de ligação entre a jiga e o DUT.



Fonte: elaborado pelo autor.

ele; sem essa ligação, os níveis lidos por ambos os lados deixam de ter significado. Na UART, as linhas são cruzadas: a transmissão de um lado vai à recepção do outro. No barramento I2C, as linhas SCL e SDA exigem resistores de *pull-up*, referenciados a 3,3 V pelas razões da subseção 3.4.3. No SPI, o sentido de SCK, MOSI e CS depende do papel da jiga: como escrava, o DUT gera esses sinais e o pino PD7 é configurado como entrada; como mestra, a jiga os gera e PD7 é configurado como saída. Por fim, no caso de um DUT de 5 V, como o Arduino Uno, valem as restrições de nível já discutidas, em especial para as entradas analógicas da jiga.

3.4.6 Lista de Materiais e Custo

A Tabela 1 apresenta os materiais que compõem a jiga e seu custo, que é o critério de aceitação do RNF01. A lista inclui apenas o que é necessário para montar e operar a jiga; os DUTs usados na validação (ESP32 e Arduino Uno), os instrumentos de medição e o computador não fazem parte dela, pois não integram o produto e são recursos que o laboratório já precisa ter para desenvolver *firmware*.

A composição da lista decorre das decisões anteriores. A placa de desenvolvimento concentra o microcontrolador, o depurador e os conectores (seção 3.4), o que dispensa a fabricação de uma placa dedicada. São necessários dois cabos USB porque o depurador ST-LINK e a porta USB OTG do microcontrolador usam conectores distintos: o primeiro alimenta e grava a placa, e o segundo transporta o canal de comandos (seção 3.6). Há dois tipos de *jumper* porque os conectores da jiga usados na ligação são fêmeas, enquanto os DUTs de referência expõem os pinos de forma diferente: o ESP32, em placas de desenvolvimento com pinos machos, e o Arduino Uno, com barras fêmeas. A *protoboard* acomoda os poucos componentes passivos

Tabela 1 – Lista de materiais da jiga de testes.

Item	**Qtd.**	**Preço unit. (R$)**	**Subtotal (R$)**
Placa de desenvolvimento NUCLEO-F767ZI, com depurador ST-LINK integrado	1	*a definir*	*a definir*
Cabo USB tipo A para micro-B (ST-LINK: alimentação e gravação)	1	*a definir*	*a definir*
Cabo USB tipo A para micro-B (porta USB OTG: canal USB-CDC)	1	*a definir*	*a definir*
Kit de cabos *jumper* macho-fêmea, 20 cm (40 unidades)	1	*a definir*	*a definir*
Kit de cabos *jumper* macho-macho, 20 cm (40 unidades)	1	*a definir*	*a definir*
Protoboard de 170 pontos	1	*a definir*	*a definir*
Resistor 4,7 kΩ, 1/4 W (*pull-up* do I2C)	2	*a definir*	*a definir*
Resistor 10 kΩ, 1%, 1/4 W (divisor de tensão)	3	*a definir*	*a definir*
Resistor 15 kΩ, 1%, 1/4 W (divisor de tensão)	3	*a definir*	*a definir*
Módulo conversor de nível lógico bidirecional, 4 canais (BSS138)	1	*a definir*	*a definir*
Caixa e tampa impressas em 3D (PLA)	1	*a definir*	*a definir*
Parafuso M3 com espaçador, para fixação da placa	4	*a definir*	*a definir*
Total			***a definir***

Fonte: elaborado pelo autor.

exigidos pela compatibilidade de níveis (subseção 3.4.3): os dois resistores de *pull-up* do barramento I2C, com valor usual para o modo padrão de 100 kHz, e um divisor de tensão por entrada analógica, que reduz um sinal de 5 V para $5 \cdot 15/(10+15) = 3{,}0$ V, abaixo do limite de 3,3 V do ADC e com margem para a tolerância dos resistores. O conversor de nível bidirecional só é necessário quando os barramentos I2C ou SPI são ligados a um DUT de 5 V. A caixa e a tampa correspondem aos modelos do projeto mecânico e protegem a placa durante o uso em bancada.

O custo total da jiga, apresentado na Tabela 1, deve ser comparado ao das plataformas comerciais de HIL discutidas na seção 2.3, cujo custo inviabiliza a adoção em laboratórios acadêmicos e pequenas equipes; essa comparação é retomada no Capítulo 4.

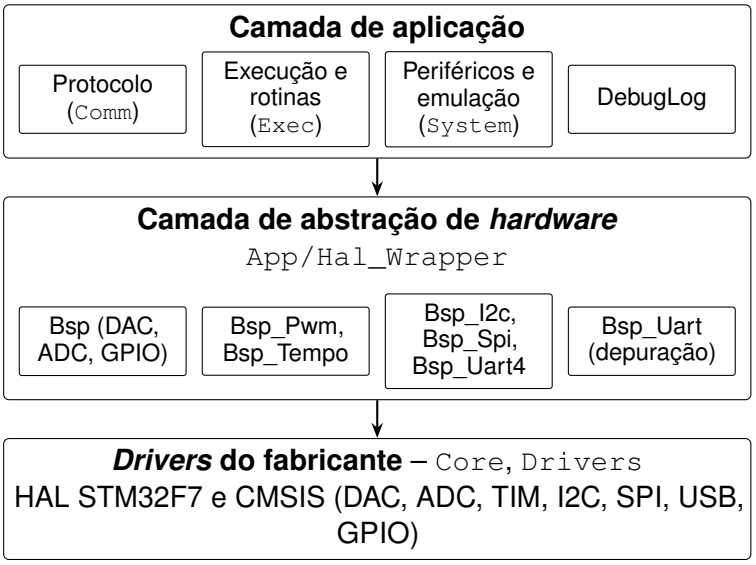
3.5 PROJETO DE FIRMWARE

O *firmware* é responsável por interpretar os comandos recebidos do computador e traduzi-los em estímulos elétricos, além de adquirir e devolver as respostas do DUT. Esta seção descreve sua arquitetura em camadas, a divisão de tarefas no sistema operacional de tempo real e a lógica de processamento de um comando, com as respectivas justificativas.

3.5.1 Arquitetura em Camadas

O código é organizado em três camadas, ilustradas na Figura 3: a camada de *drivers* do fabricante (`Core` e `Drivers`, gerados pelo STM32CubeMX, com a HAL e o CMSIS), a camada de abstração de *hardware* (`App/Hal_Wrapper`, um módulo `Bsp` por periférico) e a camada de aplicação, que se apoia no sistema operacional de tempo real FreeRTOS para organizar sua execução em tarefas. A camada de aplicação se divide em quatro grupos: o protocolo (`App/Comm`, receptor de quadros e montagem das respostas), a execução (`App/Exec`, comandos diretos, interpretador de rotinas e estado seguro), os módulos de periférico (`App/System`, com a validação dos parâmetros e a lógica do DAC, das entradas e saídas digitais, do PWM e dos barramentos, incluindo a emulação dos escravos) e o registro de depuração (`DebugLog`).

Figura 3 – Arquitetura em camadas do firmware.



Fonte: elaborado pelo autor.

A justificativa para essa separação é a mesma que fundamenta as abordagens de teste *off-target* discutidas na seção 2.2 (LINS *et al.*, 2016; BRANAM, 2025): isolando o acesso direto aos periféricos em um módulo único (`Bsp`), a camada de aplicação passa a depender de uma interface própria, e não diretamente da HAL do fabricante. Isso traz dois benefícios práticos para este trabalho. Primeiro, a portabilidade: se a jiga for futuramente portada para outro microcontrolador, apenas a camada `Bsp` precisa ser reescrita, mantendo o protocolo e a lógica de dispatch inalterados. Segundo, a testabilidade: os testes de unidade (seção 3.8) substituem o `Bsp` por periféricos simulados, e os módulos de protocolo, de execução e de periférico rodam também no computador, compilados com o `gcc`, sem depender do *hardware* da jiga.

3.5.2 Tarefas e Sincronização

Os sinais estáticos da jiga – uma tensão de DAC ou um PWM já configurados – são mantidos pelos próprios periféricos, sem intervenção do processador. O que exige concorrência é o restante: a comunicação passa a maior parte do tempo à espera de bytes da USB, a execução de um teste pode ter de aguardar o DUT reagir, e o atendimento ao DUT quando a jiga atua como escrava I2C, SPI ou UART depende do instante em que o DUT decide se comunicar, que a jiga não controla. Em um laço principal único (*superloop*), essas esperas teriam de ser convertidas manualmente em verificações periódicas de estado, e o atraso de uma atividade passaria a afetar as demais. Com um sistema operacional de tempo real, cada atividade é uma tarefa que pode bloquear à espera de um evento sem ocupar o processador, e cada responsabilidade fica isolada em um módulo. O FreeRTOS foi escolhido por já estar integrado ao STM32CubeMX, o que reduz o esforço de configuração, e por ser de código aberto, sob licença MIT.

O *firmware* é organizado nas três tarefas do Quadro 5, todas criadas na inicialização e permanentes. A `CommTask` é dedicada exclusivamente à comunicação com o computador, e a `TestExecTask` interpreta os comandos de teste e aciona os estímulos e as aquisições. A terceira, `defaultTask`, é gerada pelo STM32CubeMX: inicializa a pilha USB e, depois, apenas pisca o LED de atividade da placa.

Quadro 5 – Tarefas do FreeRTOS.

Tarefa	**Responsabilidade**	**Ciclo de vida**
`CommTask`	Comunicação com o computador pela USB OTG FS: recebe e valida os quadros (subseção 3.5.4), atende diretamente PING, ROUTINE_STATUS e ABORT, repassa os demais comandos à `TestExecTask` e envia as respostas.	Criada na inicialização; permanente.
`TestExecTask`	Executa os comandos diretos e interpreta as rotinas de teste (subseção 3.5.5): aplica os estímulos (DAC, saídas digitais, PWM e barramentos), realiza as aquisições (ADC, entradas digitais e barramentos) e aplica o estado seguro.	Criada na inicialização; permanente.
`defaultTask`	Gerada pelo STM32CubeMX: inicializa o USB Device e pisca o LED de atividade.	Criada na inicialização; permanente.

Fonte: elaborado pelo autor.

A separação entre `CommTask` e `TestExecTask` garante que uma não atrase a outra. Enquanto a `TestExecTask` executa uma rotina, a `CommTask` continua recebendo e validando quadros: responde às consultas de estado lendo as informações que a `TestExecTask` publica a cada passo, transmite um ABORT como sinal à `TestExecTask` e recusa os demais comandos com o erro OCUPADO (Quadro 14), em vez de deixar o computador esperar até o tempo limite sem saber o motivo.

No sentido inverso, a chegada de bytes pela USB não interrompe um estímulo em andamento. As duas tarefas trocam comandos e resultados por filas do FreeRTOS; como o protocolo só admite um comando pendente por vez (seção 3.6), cada fila precisa comportar apenas um elemento. As filas são criadas na configuração do STM32CubeMX, com alocação estática, pela interface CMSIS-RTOS v2. Os bytes recebidos pela USB, por sua vez, passam da interrupção à `CommTask` por um *stream buffer*, recurso da interface nativa do FreeRTOS que não existe na CMSIS-RTOS v2 nem no STM32CubeMX; o *firmware* combina, por isso, as duas interfaces, cada uma onde é necessária.

As prioridades seguem a urgência de cada atividade. O atendimento às transações iniciadas pelo DUT, que não espera, é feito nas interrupções dos periféricos e tem precedência sobre qualquer tarefa; entre as tarefas, a `TestExecTask` vem primeiro, para que a temporização dos passos de uma rotina não seja atrasada pela comunicação; e a `CommTask` fica abaixo das duas, pois só precisa responder dentro do tempo limite de 100 ms do protocolo, folga muito maior que a duração de qualquer passo isolado. Como a `TestExecTask` passa a maior parte do tempo bloqueada à espera de eventos, a `CommTask` continua sendo atendida com frequência.

Na fase de projeto, previu-se uma tarefa para cada emulação de escravo (I2C, SPI e UART), criada sob demanda. Na implementação, essas tarefas foram dispensadas. O escravo I2C é atendido nas funções de retorno da interrupção do I2C2, byte a byte; o escravo SPI, na interrupção do SPI2 e na interrupção externa do pino de seleção (PD7), que marca o início e o fim de cada transação; e a recepção da UART, na interrupção da UART4. Em todos esses casos é o DUT quem dita o tempo de cada byte, e o tratamento de cada evento se resume a poucas operações de memória; uma tarefa intermediária apenas acrescentaria uma troca de contexto entre a interrupção e o tratamento, aumentando a latência justamente onde o DUT não espera, além de consumir uma pilha própria. O critério adotado foi transformar em tarefa apenas o que precisa bloquear à espera de eventos, e organizar o restante em módulos. As interrupções dos periféricos usam a prioridade 5 do controlador de interrupções, a mais alta que ainda permite chamar as funções do FreeRTOS próprias para interrupções.

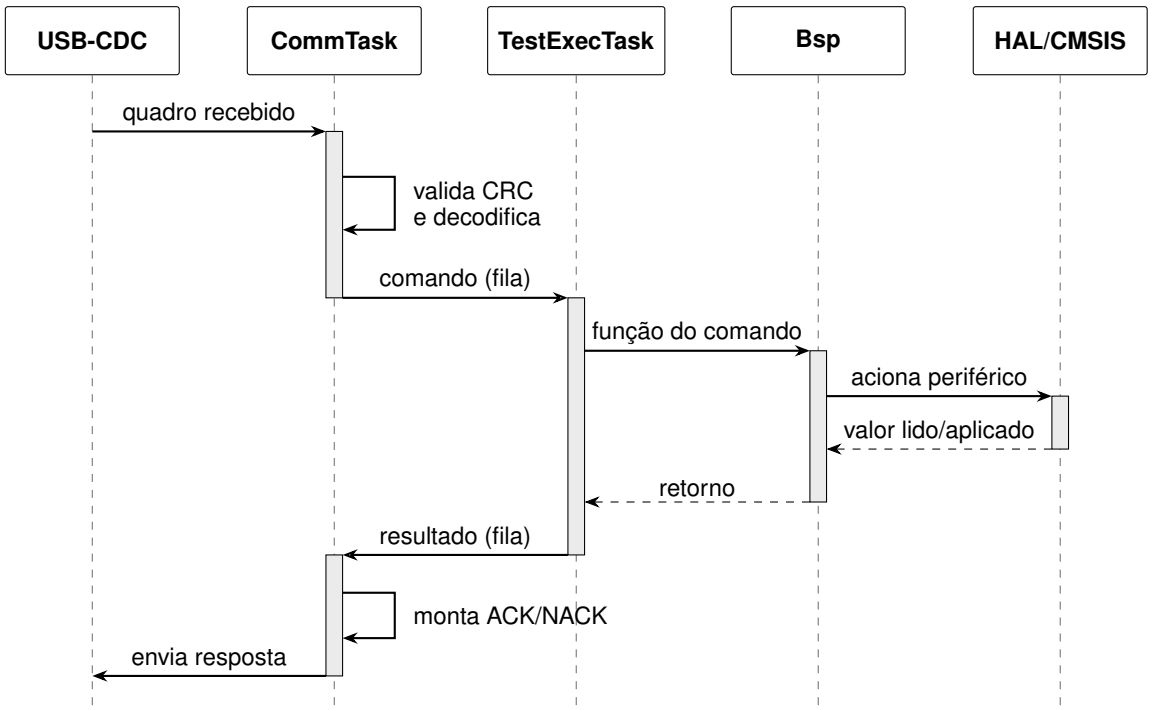
Um mutex (`uartDebugMutex`) protege o acesso à UART de depuração, evitando que mensagens de registro de tarefas distintas se intercalem.

3.5.3 Processamento de um Comando

A Figura 4 apresenta o percurso de um comando direto dentro do *firmware*, da recepção do quadro pela USB-CDC até o envio da resposta. A `CommTask` valida e decodifica o quadro e o repassa pela fila à `TestExecTask`, que executa o comando por meio do `Bsp` e devolve o resultado pela fila de retorno; só então a `CommTask` monta e envia a resposta. Entre o repasse e o retorno, a `CommTask` fica livre para continuar

atendendo a USB: ela não bloqueia à espera do resultado, e sim volta a ler a USB, com tempo limite de 20 ms, e confere a fila de resultado, sem bloquear, a cada volta. Como a `TestExecTask` tem prioridade maior, um comando rápido é executado assim que entra na fila, e o resultado já está disponível na conferência seguinte; nas medições de desempenho, um comando que passa pela `TestExecTask` levou, em média, cerca de 0,7 ms a mais que o PING, atendido pela própria `CommTask` (Capítulo 4). Só um comando demorado, como um UART_WRITE longo a uma taxa baixa, pode esperar até 20 ms a mais pela conferência, atraso pequeno diante do tempo limite de 100 ms. Um RUN_ROUTINE percorre o mesmo caminho, mas o resultado devolvido de imediato pela `TestExecTask` é apenas a aceitação da rotina, que passa então a ser executada conforme a subseção 3.5.5.

Figura 4 – Sequência de processamento de um comando direto no firmware.

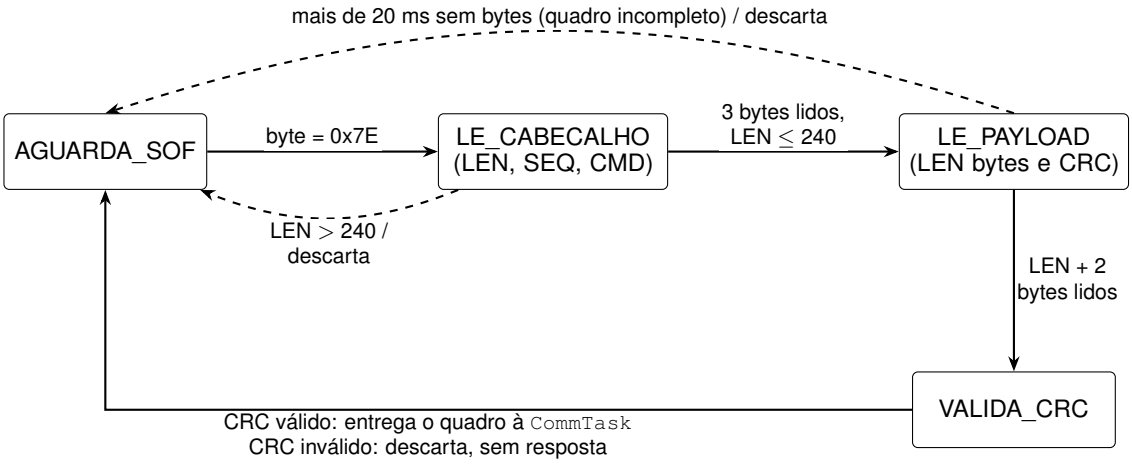


Fonte: elaborado pelo autor.

3.5.4 Máquina de Estados do Receptor de Quadros

Como o USB-CDC entrega um fluxo contínuo de bytes (seção 3.6), a `CommTask` usa um receptor de quadros implementado como uma máquina de estados finita, em vez de assumir que cada leitura da porta corresponde a um quadro completo. O receptor é um módulo à parte, sem dependência do *hardware* nem do FreeRTOS: recebe um byte por vez e devolve um quadro completo e válido, ou o motivo do descarte. A Figura 5 apresenta os estados e as transições.

Figura 5 – Máquina de estados do receptor de quadros.



Fonte: elaborado pelo autor.

A máquina de estados foi adotada, em vez de uma função bloqueante que aguarda o quadro inteiro, porque a recepção de um byte por vez, acionada pela interrupção ou pela fila da USB-CDC, não pode travar a tarefa à espera de bytes que ainda não chegaram; a máquina de estados registra o progresso da leitura e retoma de onde parou a cada novo byte recebido. O estado `VALIDA_CRC` realiza a verificação descrita na Apêndice A.3 antes de qualquer efeito sobre os periféricos, de modo que um quadro corrompido nunca chega à `TestExecTask` nem ao `Bsp`, o que atende ao RNF03.

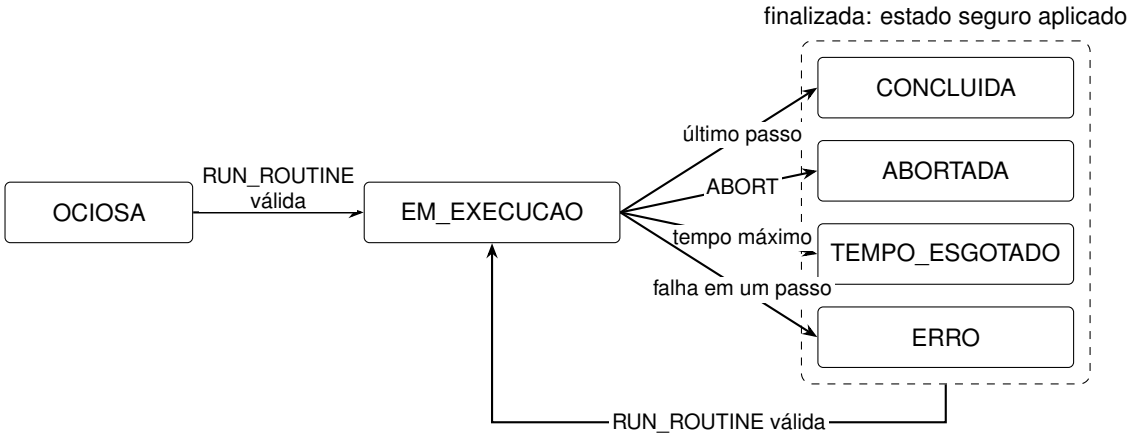
Cada quadro válido segue para a lógica de resposta da `CommTask`, separada do receptor: a retransmissão do comando que ainda está em execução é ignorada; um quadro com o SEQ do último comando respondido recebe a resposta guardada, sem nova execução (Apêndice A.7); PING, ROUTINE_STATUS e ABORT são respondidos na hora, sem passar pela `TestExecTask`, o que garante a resposta mesmo com uma rotina em execução; qualquer outro comando que chegue com um comando pendente ou durante uma rotina recebe OCUPADO; e os demais são repassados à `TestExecTask`, cuja resposta é enviada quando o resultado volta pela fila.

3.5.5 Interpretador de Rotinas

Ao receber um RUN_ROUTINE, a `TestExecTask` primeiro valida a rotina inteira, conforme a especificação do Apêndice A; só depois de validada a rotina é aceita e executada, passo a passo, na ordem em que foi escrita. O estado da execução evolui conforme a Figura 6.

A precisão temporal, que motivou a execução das rotinas na jiga (subseção 3.6.3), depende de três decisões de implementação. A primeira é a base de tempo: os instantes das medições e a duração das esperas são contados pelo temporizador TIM5, de 32 bits, configurado como contador livre a 1 MHz (108 MHz

Figura 6 – Estados de execução de uma rotina na `TestExecTask`.



Fonte: elaborado pelo autor.

divididos por 108), e não pelo relógio do sistema operacional, cuja resolução é de 1 ms (`configTICK_RATE_HZ` = 1000). O TIM5 foi escolhido por ser, ao lado do TIM2, o único temporizador de 32 bits do microcontrolador, e o TIM2 já gera o PWM2. Com 32 bits a 1 MHz, a contagem só se repete após cerca de 71 minutos, muito mais do que a duração de qualquer rotina. A segunda é a detecção de transições no passo ESPERA_GPIO: a borda na entrada monitorada é detectada por interrupção externa, cuja rotina de atendimento registra o valor do temporizador naquele momento; assim, o instante registrado não depende de quando a `TestExecTask` volta a ser escalonada. As entradas PD2 (GPIO6) e PG2 (GPIO7) compartilham a mesma linha de interrupção externa (EXTI2), como todos os pinos de número 2; como só há um passo ESPERA_GPIO ativo por vez, a linha é associada à porta da entrada monitorada no início de cada passo e liberada ao fim dele. A terceira é a forma de esperar: a tarefa fica bloqueada à espera de uma notificação, disparada pelo temporizador ao fim da espera, pela interrupção da entrada monitorada ou por um ABORT recebido pela `CommTask`. Desse modo a espera não ocupa o processador, termina com a precisão do temporizador, por meio de uma comparação do próprio TIM5, e pode ser interrompida imediatamente. Na placa, o custo de cada passo (a execução do passo e o retorno da notificação) ficou em cerca de 28 µs, dentro da tolerância temporal de 50 µs (Capítulo 4).

A cada passo concluído, a `TestExecTask` atualiza o estado publicado para a `CommTask` (estado, número de passos concluídos e código de erro) e verifica se o tempo máximo da rotina foi ultrapassado. Ao final, qualquer que seja o motivo, leva as saídas ao estado seguro e só então publica o estado final, de modo que o computador, ao ver a rotina finalizada, sabe que nenhum estímulo continua aplicado; os resultados permanecem disponíveis para leitura. Os passos são executados pelas mesmas funções dos comandos diretos, o que garante que um estímulo aplicado em

uma rotina se comporta exatamente como o comando direto equivalente, já verificado nos testes de integração.

3.5.6 Emulação de Sensores

Além de emular um dispositivo genérico (um mapa de registradores ou um *buffer*), a jiga emula dois sensores comerciais completos, o acelerômetro e giroscópio MPU-6500 e o sensor ambiental BME280 (temperatura, pressão e umidade), de modo que o DUT os lê com o seu *driver* normal, sem saber que está falando com a jiga. A emulação aproveita o mapa de 256 registradores da escrava I2C e acrescenta três recursos ao *firmware*. O primeiro é uma **máscara de bits graváveis** por registrador: um registrador só de leitura, como o de identificação ou os de dados, não muda quando o DUT escreve nele, e um bit de *reset* que no sensor real volta sozinho a zero é marcado como não gravável, de modo que o DUT escreve 1 e lê 0, como se o *reset* tivesse terminado. Sem isso, um *driver* que espera o fim do *reset* ficaria preso. O segundo é um **segundo endereço** de escrava, recurso do próprio periférico I2C do STM32F767ZI, que permite à jiga responder como os dois sensores ao mesmo tempo. Os dois endereços dividem o mesmo mapa, o que é possível porque os registradores dos dois sensores não se sobrepõem (de `0x0D` a `0x75` no MPU-6500 e de `0x88` a `0xFE` no BME280). O terceiro é o **modo registradores da escrava SPI**, que usa o mesmo mapa: em cada transação, o primeiro byte é o endereço, com o bit 7 indicando leitura, e os seguintes são os dados, com auto-incremento.

O modo SPI tem uma restrição que decorre do próprio barramento. No I2C, a escrava pode segurar a linha de *clock* (*clock stretching*) até ter o dado pronto, e o mestre espera. No SPI, o mestre gera o *clock* sem esperar a escrava, e o primeiro dado precisa começar a sair logo após o último bit do endereço, que é justamente quando a jiga descobre qual registrador foi pedido. Por isso, o mestre precisa deixar uma pausa de alguns microssegundos entre o byte de endereço e o primeiro dado; daí em diante, a interrupção do SPI mantém a FIFO de transmissão de 4 bytes cheia com os registradores seguintes, e a leitura segue na velocidade do mestre. Um mestre que emenda o endereço e os dados sem pausa, como os que usam DMA, recebe o primeiro dado errado. A restrição foi aceita e documentada, porque cobri-la exigiria um circuito externo dedicado; como a jiga tem um único pino de seleção, apenas um sensor por vez pode ficar no SPI, enquanto o outro pode ficar no I2C.

O *firmware* não conhece os sensores: ele só tem o mapa, as máscaras e os endereços. Os modelos dos sensores ficam na aplicação, que converte as grandezas físicas nos valores dos registradores. No MPU-6500, a aceleração, a rotação e a temperatura são inteiros de 16 bits com sinal, proporcionais ao fundo de escala configurado. No BME280, o DUT não lê a grandeza diretamente: lê valores brutos e os compensa com os coeficientes de calibração do sensor, gravados no próprio

mapa, segundo as fórmulas da folha de dados. A aplicação calcula a operação inversa, encontrando por busca binária o valor bruto que, depois da compensação, resulta na temperatura, na pressão e na umidade pedidas, com erro de até 0,005 nas três grandezas. A configuração inicial de um sensor, com os valores de *reset*, a calibração e as máscaras, passa de 100 bytes e não cabe junto com os passos de um caso nos 240 bytes de uma rotina; ela é enviada como um **preparo** do caso, em comandos diretos antes do RUN_ROUTINE, e a rotina contém apenas as mudanças de valor.

3.5.7 Gerador de Ondas

Muitos comportamentos de interesse dependem de como um sinal varia no tempo: um termostato que liga ao passar de um limiar durante um aquecimento, um filtro que deve ignorar uma vibração, um controle que reage a uma rampa. Para isso, a jiga tem um gerador de ondas que varia um valor sozinho, em segundo plano, enquanto a rotina continua; assim, as esperas e as verificações do caso acontecem com a onda em andamento. Os destinos possíveis são os dois canais do DAC, a razão cíclica dos três canais de PWM e os registradores dos sensores emulados, e até seis ondas podem rodar ao mesmo tempo.

O *firmware* conhece apenas uma tabela de 2 a 32 pontos igualmente espaçados no período, com interpolação linear ou em degraus, um número de ciclos (zero significa até o fim da rotina) e um ruído uniforme opcional; a forma da onda é montada pela aplicação. Com isso, qualquer forma custa o mesmo para o *firmware*, e a aplicação oferece rampa, senoide, triangular, quadrada, dente de serra, exponencial (a resposta de um sistema de primeira ordem, como um aquecimento real) e uma sequência livre de pontos. Uma senoide com 32 pontos e interpolação linear se afasta menos de 0,5% da amplitude da senoide verdadeira, e as formas triangular, dente de serra e rampa são exatas.

A atualização dos valores depende do passo da onda. Com passos de 100 µs ou mais, cada onda é atualizada em uma interrupção agendada no canal 2 de comparação do TIM5, a mesma base de tempo das rotinas, para o próximo passo da onda mais próxima. O passo padrão de 1 ms foi escolhido porque, nos sensores, uma atualização mais rápida não muda o que o DUT observa: o próprio BME280 entrega no máximo cerca de 180 medidas por segundo, e os *drivers* costumam ler o MPU-6500 a até 1 kHz. Para o DAC, porém, 1 ms limitaria uma senoide a algumas dezenas de hertz. Nesse caso, com passos de 1 a 99 µs, um período de até 1024 amostras é calculado e tocado pelo DMA, com o disparo do DAC por um temporizador (TIM7 para o DAC1 e TIM4 para o DAC2), sem ocupar o processador. Fazer o processador atualizar o DAC a cada poucos microssegundos roubaria tempo das interrupções que atendem o DUT nos barramentos, que precisam responder na hora. A onda na razão cíclica do PWM altera apenas o registrador de comparação do temporizador, sem reiniciar a contagem,

para não distorcer o período. Ao fim da rotina, e também no ABORT e no RESET_ALL, todas as ondas são paradas antes de as saídas irem ao estado seguro.

Uma onda na temperatura do BME280 tem um cuidado adicional: como a compensação da pressão e da umidade depende da temperatura, mudar só o valor bruto da temperatura faria o DUT ler também uma pressão e uma umidade diferentes. A aplicação acrescenta, por isso, duas ondas que corrigem os valores brutos da pressão e da umidade ao longo da rampa, calculadas pelo mesmo modelo inverso.

3.6 PROTOCOLO DE COMUNICAÇÃO PC–JIGA

A comunicação entre a aplicação *desktop* e o *firmware* é feita por um protocolo próprio de comandos, transportado sobre USB-CDC, que o computador enxerga como uma porta serial virtual. A especificação completa, com formato do quadro, códigos de comando, erros e exemplos, está no Apêndice A. Esta seção resume as escolhas e as justifica.

3.6.1 Meio de Transmissão

A placa oferece três caminhos possíveis entre o computador e o microcontrolador: a UART ligada ao depurador integrado (USART3), que chega ao computador por meio do conversor USB-serial do próprio depurador; a porta Ethernet; e o periférico USB OTG *full-speed* do STM32F767ZI, operando na classe CDC (*Communications Device Class*). Escolheu-se o USB-CDC por três razões. Primeiro, ele usa um periférico do próprio microcontrolador, e o computador o reconhece como porta serial virtual com os *drivers* nativos dos sistemas operacionais atuais, de modo que a aplicação o acessa com a mesma biblioteca serial que usaria para uma UART. Segundo, a vazão não fica presa à taxa de *baud* configurada: os dados trafegam em pacotes USB *full-speed*, de 12 Mbit/s. Terceiro, em relação à Ethernet, dispensa configuração de rede, endereçamento IP e uma pilha TCP/IP no *firmware*, que aumentariam a complexidade sem benefício para uma ligação ponto a ponto com um único computador. A UART do depurador fica, assim, reservada ao registro de depuração (`DebugLog`), o que separa as mensagens de diagnóstico do canal de comandos.

3.6.2 Modelo de Comunicação

O protocolo segue o modelo requisição–resposta: o computador inicia toda a comunicação e cada comando recebe exatamente uma resposta, ACK ou NACK, o que atende ao RF06. Adotou-se também que o computador só envia um novo comando depois de receber a resposta do anterior. Essa escolha simplifica o *firmware*, que dispensa fila de comandos e controle de fluxo. O custo é uma vazão menor, que não é

limitante aqui, pois o tempo de um teste é dominado pela resposta do DUT e não pela taxa da USB.

Há dois tipos de comando. Os **comandos diretos** executam uma única operação – aplicar uma tensão no DAC, ler uma entrada digital – e servem à operação manual da jiga e aos testes de integração. Os **comandos de rotina** transferem à jiga um caso de teste completo, isto é, uma sequência de estímulos, esperas e aquisições, que a jiga executa sozinha (subseção 3.6.3).

3.6.3 Rotinas Executadas na Jiga

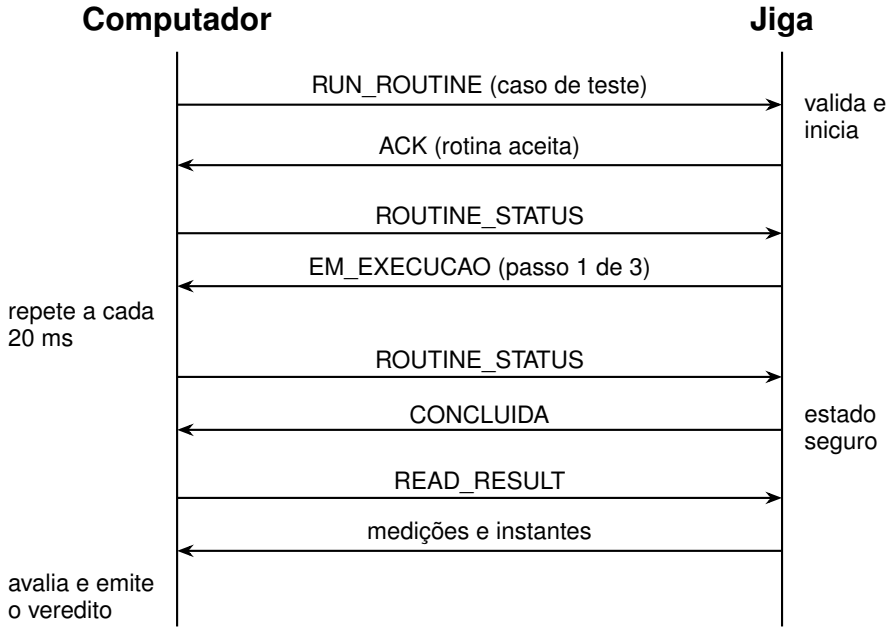
Cada caso de teste definido pelo usuário na aplicação é convertido em uma rotina e enviado em um único comando, RUN_ROUTINE; uma bateria de quinze casos corresponde, assim, a quinze comandos, enviados um de cada vez. A execução da rotina na própria jiga, em vez de o computador comandar cada estímulo e cada leitura separadamente, foi adotada principalmente pela precisão temporal. Se os passos fossem comandados pelo computador, o intervalo entre um estímulo e a leitura seguinte incluiria a ida e volta pela USB e o escalonamento do sistema operacional do computador, que variam de uma execução para outra na escala de milissegundos. Executados na jiga, os passos são temporizados pelo relógio do microcontrolador, e cada medição é registrada com o instante em que ocorreu, com resolução de microssegundos (subseção 3.5.5); isso torna possível verificar tempos de reação do DUT e aumenta a reprodutibilidade (RNF04). A rotina executada na jiga também é mais robusta: se a comunicação cair no meio de um caso, a jiga conclui a rotina e volta sozinha ao estado seguro, em vez de deixar um estímulo aplicado pela metade.

A granularidade de um caso por comando, e não de uma bateria inteira, foi escolhida porque cada caso tem o seu próprio veredito, a jiga retorna ao estado seguro entre um caso e outro, uma falha em um caso não compromete os seguintes, e a aplicação pode informar o progresso ao usuário. Uma bateria inteira em um só comando, além disso, não caberia no tamanho máximo de um quadro.

Como uma rotina pode durar vários segundos, mais do que o tempo limite de resposta do protocolo, o ACK de RUN_ROUTINE é enviado assim que a rotina é validada e começa a ser executada, e não ao seu término. Enquanto a rotina é executada, o computador consulta periodicamente o estado com ROUTINE_STATUS e, ao final, lê as medições com READ_RESULT, conforme a Figura 7.

O acompanhamento por consultas, em vez de uma mensagem espontânea da jiga ao terminar, mantém o computador como único a iniciar a comunicação. Assim, o mesmo mecanismo de tempo limite e retransmissão vale para todas as mensagens, e o computador nunca precisa distinguir uma resposta de uma notificação não solicitada. O tráfego adicional, de quadros de poucos bytes a cada 20 ms, é desprezível diante da capacidade do USB. Durante a execução, a jiga atende apenas PING, ROU-

Figura 7 – Ciclo de execução de uma rotina entre o computador e a jiga.



Fonte: elaborado pelo autor.

TINE_STATUS e ABORT; qualquer outro comando recebe NACK OCUPADO, o que impede que um comando direto interfira em uma rotina em andamento.

Duas proteções limitam a duração de uma rotina. O comando ABORT permite ao usuário interromper a execução a qualquer momento, inclusive durante uma espera, e cada rotina informa um tempo máximo, fiscalizado pela própria jiga, que a interrompe se ele for ultrapassado. Nos dois casos, e também ao término normal, a jiga volta ao estado seguro (RNF05).

Por fim, a jiga devolve apenas as medições, sem avaliá-las; o veredito é emitido pelo computador. Com isso, o interpretador do *firmware* permanece simples, os critérios e as tolerâncias ficam em unidades físicas no arquivo da bateria de testes (subseção 3.7.3) e o relatório preserva os dados brutos de cada execução (RF09).

3.6.4 Formato do Quadro

Como o USB-CDC entrega um fluxo contínuo de bytes, sem delimitar mensagens, cada mensagem é encapsulada em um quadro composto por um byte de início (SOF), o tamanho dos dados (LEN), um número de sequência (SEQ), o código do comando (CMD), os dados (PAYLOAD) e uma verificação de integridade (CRC-16), conforme a Figura 13. Optou-se por um formato **binário**, e não por texto (como comandos ASCII terminados em quebra de linha), porque o cabeçalho de tamanho fixo permite ao *firmware* decodificar o quadro com pouco processamento e sem análise de *strings*, e porque o lado Python o monta e desmonta diretamente com a biblioteca padrão. A desvantagem, a leitura menos direta em um terminal serial, é atenuada pelo

registro de quadros da própria aplicação e pelos exemplos do apêndice.

O enquadramento é feito pelo comprimento: o byte `0x7E` só marca o início de um quadro quando o receptor está procurando o SOF, e dentro do quadro é um byte comum, delimitado pelo campo LEN. A alternativa usual, a inserção de bytes de escape (*byte stuffing*), garantiria que o `0x7E` só aparecesse como delimitador, mas faria o tamanho do quadro depender do conteúdo – um *payload* de 240 bytes poderia chegar a quase o dobro – e exigiria codificar e decodificar cada byte nos dois lados. O risco do enquadramento por comprimento é a perda de alinhamento quando um byte se perde ou um LEN chega corrompido; ele é coberto pelo CRC, que descarta o quadro mal formado, pelo tempo limite de 20 ms entre bytes, que descarta um quadro incompleto e devolve o receptor à busca do SOF, e pela retransmissão do computador.

3.6.5 Detecção de Erros e Retransmissão

Embora o próprio USB já proteja a transferência no nível físico, o protocolo inclui um CRC-16 porque o fluxo de bytes pode perder o alinhamento das mensagens (por exemplo, com *buffers* truncados ou reinício de um dos lados), e um comando aplicado a partir de bytes desalinhados poderia gerar um estímulo errado sem que ninguém percebesse, o que comprometeria a validade do teste (RNF03). O CRC-16/CCITT foi escolhido por ser amplamente utilizado, de baixo custo computacional e adequado ao tamanho máximo do quadro (246 bytes).

Quando a resposta não chega ou chega corrompida, o computador retransmite o comando com o mesmo SEQ. A jiga guarda a última resposta e, ao receber um SEQ repetido, reenvia essa resposta sem executar o comando de novo. Com isso a retransmissão é segura mesmo para comandos com efeito, como a escrita de um registrador I2C, que não são aplicados duas vezes, e um RUN_ROUTINE retransmitido não reinicia a rotina em andamento.

3.6.6 Comandos e Papéis nos Barramentos

Os comandos diretos estão organizados por periférico: sistema, DAC, GPIO, PWM, ADC, I2C, SPI e UART, cobrindo os requisitos funcionais RF01 a RF05. Os passos de uma rotina reaproveitam a mesma codificação desses comandos, acrescida de dois passos que só fazem sentido dentro de uma rotina, por dependerem de tempo: uma espera de duração fixa e uma espera por um nível em uma entrada digital, que registra o instante da transição. A principal aplicação da jiga é fornecer ao DUT os dados que um sensor real forneceria; por isso, nos barramentos I2C e SPI a jiga atua, por padrão, como **escrava**: ela mantém um mapa de registradores (I2C) ou *buffers* de transmissão e recepção (SPI), que emulam o sensor, e o DUT a consulta como faria com o dispositivo real. O papel de mestre também é oferecido, para o caso em que o DUT é quem responde, e a escolha do papel é feita por um comando de configuração.

Além disso, a jiga leva todas as saídas a um estado seguro na inicialização, sob o comando RESET_ALL e ao final de cada rotina, para que estímulos de uma execução anterior não permaneçam aplicados ao DUT (RNF05).

3.7 APLICAÇÃO DESKTOP

3.7.1 Escolha da Linguagem

A aplicação *desktop* concentra a criação das rotinas de teste, a comunicação com a jiga e o veredito dos casos. Como toda a comunicação com a jiga se dá por uma porta serial virtual (USB-CDC), o *firmware* e o protocolo independem da linguagem escolhida para o computador, e a decisão se resume às demandas da camada de controle e automação. Foram consideradas Python, C# e C++.

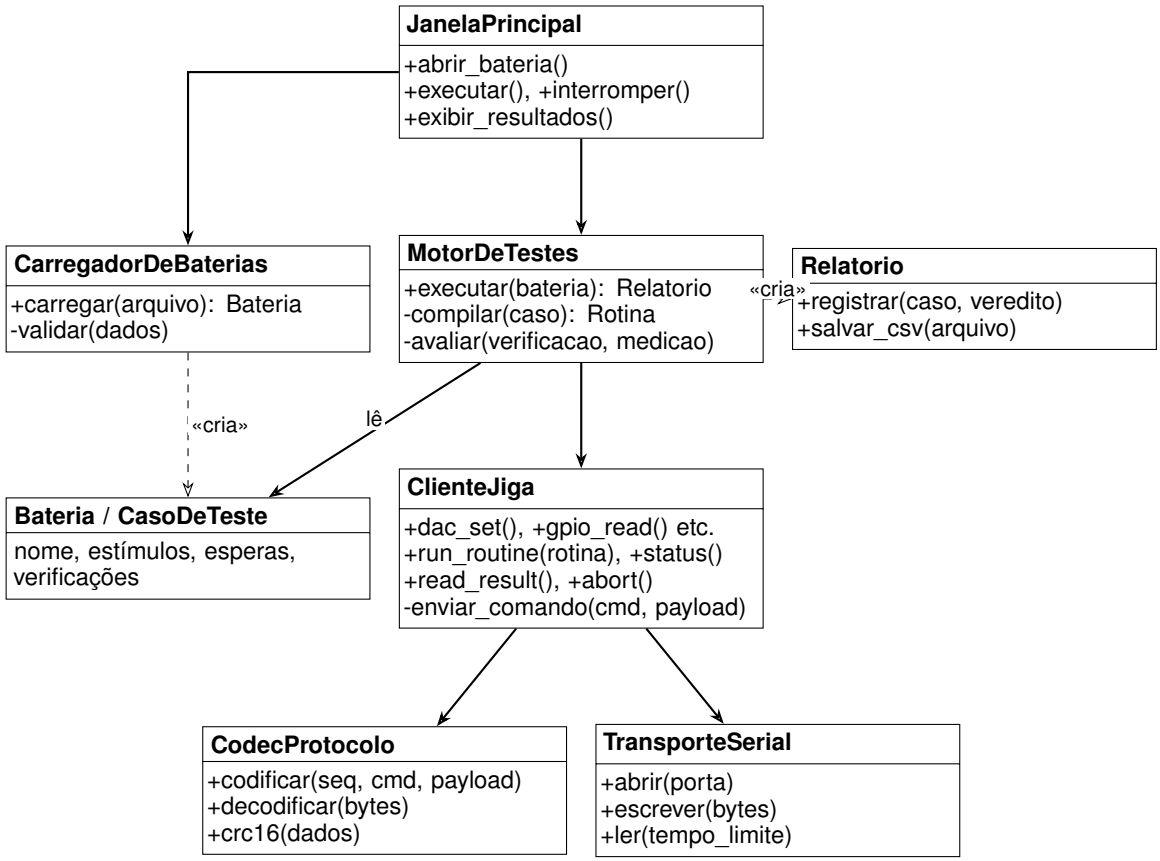
A linguagem escolhida foi o **Python**, pelos seguintes motivos: (i) a comunicação serial é resolvida por bibliotecas maduras, sem necessidade de código específico de plataforma; (ii) a execução automática de rotinas de teste é o núcleo da aplicação, e a linguagem facilita tanto a interpretação dos arquivos de rotina quanto os testes automatizados da própria aplicação (com o *pytest*), sem depender de *frameworks* externos de orquestração como o Robot Framework, cujas alternativas são discutidas por TheEmbeddedKit (2024); (iii) a geração de gráficos e relatórios das respostas adquiridas dispõe de bibliotecas prontas; e (iv) o desempenho do computador não é o fator limitante, pois o tempo de um teste é dominado pela comunicação USB e pela resposta do DUT, e não pelo processamento no computador. C# ofereceria uma interface gráfica nativa mais simples no Windows e distribuição em um único executável, mas restringiria a aplicação a essa plataforma e tornaria menos direta a escrita de rotinas como código. C++ não traria ganho perceptível para este contexto e exigiria bibliotecas adicionais de comunicação serial e interface, com maior custo de desenvolvimento.

3.7.2 Arquitetura da Aplicação

A aplicação segue a mesma ideia de camadas do *firmware*, organizada nas classes da Figura 8. Na base, `TransporteSerial` abre a porta serial virtual e troca bytes com a jiga, usando a biblioteca *pyserial*. Acima dela, `CodecProtocolo` monta e decodifica os quadros e calcula o CRC, exatamente como especificado no Apêndice A. `ClienteJiga` oferece uma operação por comando do protocolo, tanto dos comandos diretos (`dac_set`, `gpio_read` etc.) quanto dos comandos de rotina (`run_routine`, `status`, `read_result` e `abort`), e concentra o controle do número de sequência, do tempo limite e das retransmissões, de modo que as camadas superiores não precisam conhecer o formato dos quadros. No topo, `CarregadorDeBaterias` lê e valida os arquivos que descrevem os testes, `MotorDeTestes` converte cada caso de teste em uma rotina, acompanha sua execução na jiga e emite os vereditos a partir das medições

devolvidas, `Relatorio` registra os resultados e `JanelaPrincipal` forma a interface gráfica.

Figura 8 – Diagrama de classes da aplicação *desktop*.



Fonte: elaborado pelo autor.

Essa divisão foi adotada por três motivos. O primeiro é a testabilidade: como `ClienteJiga` depende de `TransporteSerial` apenas pela sua interface, os testes de unidade substituem o transporte por uma versão simulada, que pode, por exemplo, descartar respostas de propósito para verificar a retransmissão, sem precisar da jiga (subseção 3.8.3). O segundo é a separação entre interface e lógica: o motor de testes não depende da janela gráfica, de modo que uma mesma bateria pode ser executada tanto pela interface quanto por linha de comando, o que permite rodar testes de regressão de forma automática a cada nova versão do *firmware* do DUT. O terceiro é o isolamento do protocolo: se o formato do quadro mudar, apenas `CodecProtocolo` é alterado. A interface gráfica é construída com a biblioteca PySide6 (Qt), escolhida por ser multiplataforma, por ter licença LGPL e por oferecer componentes prontos de tabela e de gráfico para exibir os resultados.

Na implementação, as classes da figura correspondem a módulos Python de mesmo papel. O `CodecProtocolo` é o módulo do protocolo, que reúne as funções de montagem dos quadros e o receptor, o mesmo usado pelos testes de integração. Ao lado do `TransporteSerial` existe um transporte simulado, ligado a uma jiga

simulada que espelha o *firmware*; ele pode descartar respostas de propósito e avança o relógio da jiga simulada nas esperas, de modo que o cliente, o motor de testes e a própria janela são testados sem a placa. O cliente usa um tempo limite de 100 ms e três tentativas por comando, e a janela executa o motor de testes em uma *thread* própria, para que a interface continue respondendo durante a execução e o botão de interromper tenha efeito imediato.

3.7.3 Descrição dos Testes

Os testes são descritos em um arquivo de texto no formato YAML, chamado aqui de **bateria de testes**, composto por **casos de teste**. Cada caso define os estímulos a aplicar, as esperas e as verificações a fazer sobre as respostas do DUT, e dá origem a uma rotina executada pela jiga (subseção 3.6.3). A Figura 9 ilustra uma bateria para um *firmware* hipotético que deve ligar uma saída quando a tensão de um sensor, simulada pelo DAC, ultrapassa um limiar. O primeiro caso verifica também o tempo de reação: a saída do DUT deve ir a nível alto em até 100 ms após o estímulo.

Figura 9 – Exemplo de bateria de testes.

```
bateria: controle-ventilador
dut: ESP32
casos:
  - nome: liga acima do limiar em até 100 ms
    estimulos:
      - dac: {canal: 1, tensao: 2.0}
    verificar:
      - reacao_gpio: {pino: 5, nivel: 1, max_ms: 100}
  - nome: desliga abaixo do limiar
    estimulos:
      - dac: {canal: 1, tensao: 1.0}
    espera_ms: 500
    verificar:
      - gpio_in: {pino: 5, esperado: 0}
      - adc: {canal: 0, esperado: 1.65, tolerancia: 0.05}
```

Fonte: elaborado pelo autor.

A descrição declarativa, em vez de testes escritos como código, foi escolhida para que um usuário sem conhecimento de Python consiga criar e ler uma bateria, e para que as baterias possam ser versionadas junto com o *firmware* do DUT, apoiando os testes de regressão a cada alteração (RF07). O YAML foi preferido ao JSON por admitir comentários e ser mais legível para arquivos editados à mão.

Antes da execução, a aplicação compila cada caso em uma rotina binária. Cada estímulo vira o passo correspondente, com as grandezas convertidas de unidades físicas (volts, hertz, porcentagem) para os valores brutos do protocolo – por exemplo, uma tensão V no DAC de 12 bits corresponde ao código $\mathrm{round}(4095 \cdot V / 3{,}3)$ –, o

que evita erros de conversão manual. Cada espera vira um passo ESPERA; cada verificação vira o passo de aquisição correspondente; e a verificação de tempo de reação vira um passo ESPERA_GPIO, cujo registro já traz o tempo decorrido. Os valores esperados e as tolerâncias não são enviados à jiga: permanecem na aplicação, que os usa para avaliar as medições devolvidas. Cada verificação analógica traz sua própria tolerância, pois a margem aceitável depende do que o *firmware* do DUT deve distinguir. O tempo máximo de cada rotina é calculado pela aplicação a partir da soma das esperas, dos tempos limite e do tempo de transmissão pela UART do caso, multiplicada por 1,1 e acrescida de 100 ms; o arquivo pode também fixar esse valor em cada caso. O Quadro 6 resume os itens admitidos em uma bateria.

Quadro 6 – Itens de uma bateria de testes.

Item	**Campos**	**Passo gerado e critério**
Estímulos		
`dac`	canal, tensão em V (ou código)	DAC_SET
`gpio_out`	pino 1–4, nível	GPIO_SET
`pwm`, `pwm_stop`	canal, frequência em Hz, razão em %	PWM_SET, PWM_STOP
`espera_ms`, `espera_us`	duração	ESPERA
`i2c_config`, `i2c_load`	papel, endereço, velocidade; *offset* e dados	I2C_CONFIG, I2C_SLAVE_LOAD
`spi_config`, `spi_load`	papel, modo, frequência; dados	SPI_CONFIG, SPI_SLAVE_LOAD
`uart_config`, `uart_write`	taxa; texto ou bytes	UART_CONFIG, UART_WRITE
`mpu6500`, `bme280`	aceleração, rotação e temperatura; temperatura, pressão e umidade (sensores da seção `sensores`)	I2C_SLAVE_LOAD dos registradores de dados
`onda`	alvo, forma (rampa, senoide, triangular, quadrada, dente de serra, exponencial ou pontos), período ou duração, ciclos, passo e ruído	ONDA (uma ou mais)
Verificações		
`gpio_in`	pino 5–8, nível esperado	GPIO_READ; igualdade
`adc`	canal, tensão esperada, tolerância (padrão de 33 mV)	ADC_READ; dentro da tolerância
`reacao_gpio`	pino 5–8, nível, tempo máximo em ms	ESPERA_GPIO; transição antes do limite
`uart_read`	texto ou bytes esperados	UART_READ; igualdade
`i2c_dump`, `i2c_xfer`	*offset* ou endereço; bytes esperados	I2C_SLAVE_DUMP, I2C_MASTER_XFER; igualdade
`spi_read`, `spi_xfer`	bytes esperados	SPI_SLAVE_READ, SPI_MASTER_XFER; igualdade
`uart_valores`	comando de texto ao DUT, números esperados e tolerância	UART_WRITE, ESPERA e UART_READ; cada número dentro da tolerância

Fonte: elaborado pelo autor.

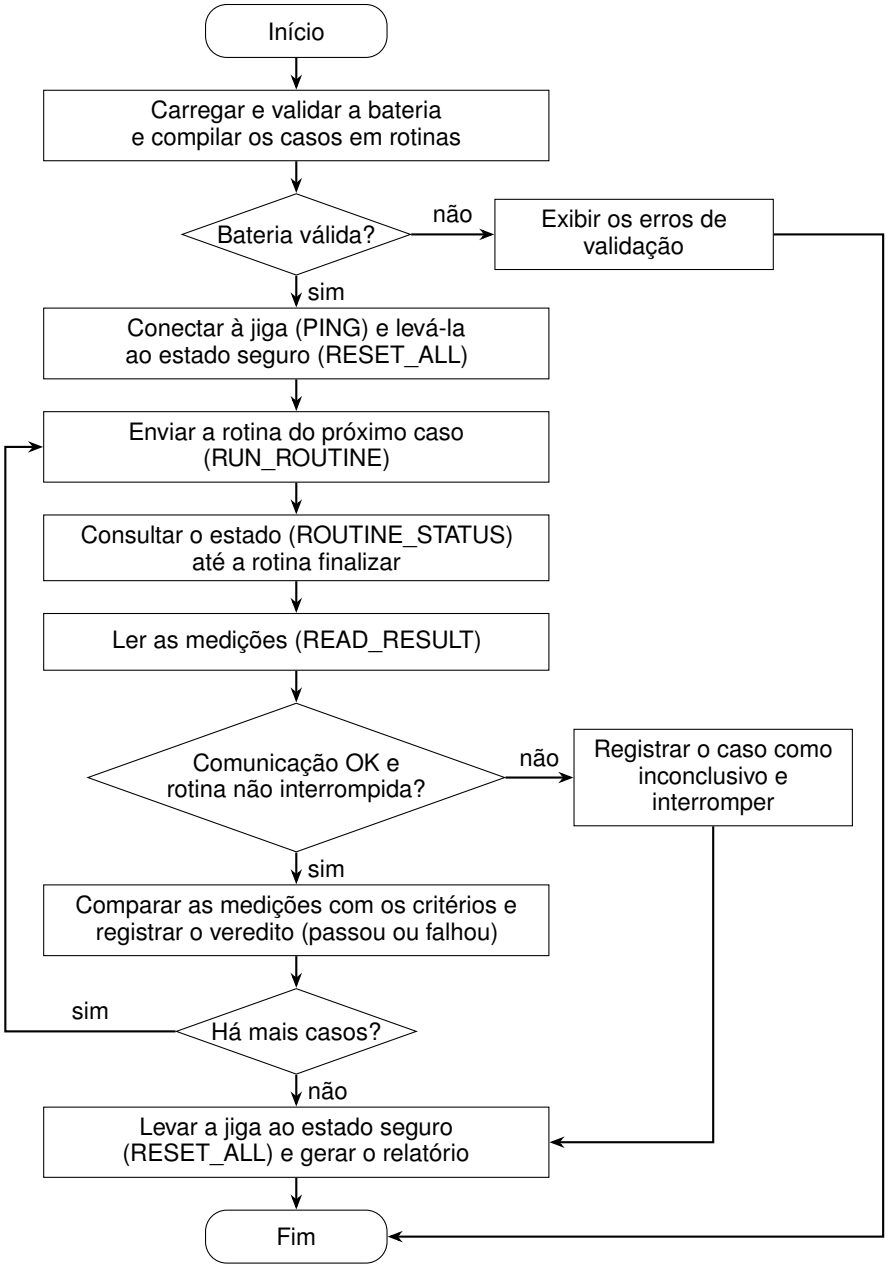
Como cada rotina termina no estado seguro, cada caso configura os periféricos que usa, sem depender do caso anterior. A bateria inteira é validada antes da

execução, e todos os erros encontrados são informados de uma vez, cada um com o caso e o item correspondentes; entre as validações estão as mesmas que a jiga aplica à rotina, de modo que uma rotina recusada pela jiga indicaria um defeito da própria aplicação, e não da bateria.

3.7.4 Execução de uma Bateria

A Figura 10 apresenta a lógica com que o `MotorDeTestes` executa uma bateria.

Figura 10 – Fluxograma de execução de uma bateria de testes.



Fonte: elaborado pelo autor.

Três decisões dessa lógica merecem justificativa. Primeiro, todos os casos

são compilados e validados antes da primeira comunicação com a jiga, de modo que um erro de descrição em um dos últimos casos não seja descoberto apenas depois de metade da bateria executada. Segundo, a jiga é levada ao estado seguro no início e no fim da bateria, além de retornar a ele sozinha ao término de cada rotina, para que nenhum estímulo de uma execução anterior influencie a atual nem permaneça aplicado ao DUT (RNF04 e RNF05). Terceiro, além de “passou” e “falhou”, existe o veredito “inconclusivo”, atribuído quando a comunicação com a jiga falha mesmo após as retransmissões ou quando a rotina é interrompida, por ABORT ou por ultrapassar o tempo máximo. A distinção é necessária porque, nesses casos, não se sabe se todos os estímulos foram aplicados, e registrar uma falha do DUT seria atribuir a ele um problema que pode ser da própria jiga ou da bateria; por isso a execução é interrompida, em vez de prosseguir com resultados de validade duvidosa (RNF03). Uma rotina finalizada em ERRO, por outro lado, indica que o DUT não respondeu em um barramento, o que é comportamento do próprio DUT: o caso é avaliado normalmente e registrado como falho, com o código do erro. O usuário também pode interromper a bateria pela interface, o que envia ABORT à jiga; o caso em andamento é registrado como inconclusivo, e os seguintes não são executados.

3.8 ESTRATÉGIA DE VERIFICAÇÃO

A verificação da solução foi planejada em quatro níveis funcionais, escolhidos de acordo com o que cada camada permite comprovar, complementados por testes de desempenho. Esta seção define os níveis, os critérios de aceitação e os casos de teste; os dados obtidos e sua análise são apresentados no Capítulo 4.

3.8.1 Níveis de Teste

Testes de unidade. Verificam isoladamente as funções de codificação e decodificação do protocolo (quadro, campos, soma de verificação), a compilação dos casos em rotinas e o motor de testes da aplicação. Por não dependerem do *hardware*, podem ser executados no computador a cada alteração, o que segue a abordagem de teste *off-target* descrita por Branam (2025) e por Lins *et al.* (2016). Cobrem RF06, RF07, RF08, RF10 e RNF03.

Testes de integração. Verificam a comunicação entre a aplicação e o *firmware* por USB-CDC, o funcionamento de cada periférico da jiga e a execução de rotinas, inclusive as regras de ocupação, interrupção e tempo máximo e a precisão temporal. Para isso, a jiga é testada em *loopback*, sem DUT, ligando por *jumpers* suas próprias saídas às suas entradas: DAC às entradas analógicas, saídas digitais às entradas digitais e MOSI a MISO. Esse arranjo permite medir se o que a jiga gera é o que ela adquire, e aproveita a ideia de expor pontos de teste desde o projeto (Altium

Resources, 2024). Cobrem RF01 a RF06, RF10, RNF03 e RNF05.

Testes de sistema. Executam baterias completas com a jiga ligada a um DUT cujo comportamento é conhecido. Verificam a interação entre estímulo, resposta e veredito. Cobrem RF04, RF08, RF09, RF10 e RNF04.

Testes de aceitação. Validam *black-box* (Portal Embarcados, 2020b) a solução em pelo menos dois microcontroladores-alvo distintos, com *firmware* de comportamento previamente conhecido, comparando os resultados obtidos com os esperados. Cobrem RNF02 e o objetivo específico (e) do Capítulo 1.

O Quadro 7 relaciona os requisitos aos níveis de teste que os comprovam.

Quadro 7 – Rastreabilidade entre requisitos e níveis de verificação.

Requisito	Unidade	Integração	Sistema	Aceitação	Desempenho
RF01 a RF03		•			
RF04		•	•		
RF05		•			
RF06	•	•			
RF07	•				
RF08	•		•	•	
RF09			•		
RF10	•	•	•		
RF11	•	•	•		
RF12	•	•	•		
RNF01	análise da lista de materiais				
RNF02				•	
RNF03	•	•			•
RNF04			•		
RNF05		•	inspeção do projeto (subseção 3.4.3)		
RNF06	inspeção do projeto				

Fonte: elaborado pelo autor.

3.8.2 Critérios de Aceitação

Os critérios numéricos adotados para as grandezas analógicas e temporais estão no Quadro 8. Para as grandezas digitais – níveis lógicos e dados transmitidos pelos barramentos – o critério é a igualdade exata, pois qualquer diferença indica erro.

A tolerância analógica de 1% do fundo de escala é cerca de quarenta vezes maior que a resolução do DAC e do ADC de 12 bits ($3{,}3\ \mathrm{V}/4096 \approx 0{,}8\ \mathrm{mV}$). A margem é intencional: ela absorve os erros de desvio (*offset*) e de ganho dos conversores e a incerteza do instrumento de referência, que a resolução sozinha não considera, e ainda é pequena diante das diferenças de tensão que um teste de emulação de sensor normalmente precisa distinguir. Quando um caso exigir precisão maior, a tolerância de cada verificação pode ser reduzida no próprio arquivo da bateria (subseção 3.7.3). Para o PWM, a frequência deriva do oscilador da placa e é quantizada pelos divisores do temporizador; o limite de 1% é folgado para ambos os efeitos nas frequências de

Quadro 8 – Critérios de aceitação das grandezas medidas.

Grandeza	Critério	Requisito
Tensão de saída do DAC	$\lvert V_{\text{medida}} - V_{\text{comandada}} \rvert \leq 33$ mV (1% do fundo de escala)	RF01
Leitura do ADC	$\lvert V_{\text{lida}} - V_{\text{aplicada}} \rvert \leq 33$ mV (1% do fundo de escala)	RF05
Frequência do PWM	desvio de até 1% do valor comandado	RF03
Razão cíclica do PWM	desvio de até 1 ponto percentual	RF03
Níveis digitais e dados de barramento	igualdade exata	RF02, RF04, RF05
Instantes registrados, tempos de reação e durações de espera em uma rotina	$\lvert t_{\text{jiga}} - t_{\text{osciloscópio}} \rvert \leq 50$ µs	RF10
Latência de ida e volta de um comando	máxima inferior a 50 ms	RNF03

Fonte: elaborado pelo autor.

interesse. A tolerância temporal de 50 µs é cinquenta vezes maior que a resolução da base de tempo (1 µs), para acomodar a latência das interrupções e o tempo de execução de cada passo, e vinte vezes menor que a resolução de 1 ms do relógio do sistema operacional, que seria a precisão obtida sem a base de tempo dedicada (subseção 3.5.5). O limite de latência corresponde à metade do tempo limite de resposta do protocolo (100 ms, Apêndice A.7), para que atrasos normais nunca provoquem retransmissões indevidas.

Os critérios do DAC e do ADC valem na faixa útil do DAC, de 0,1 a 3,2 V (subseção 3.4.4). Em códigos do ADC, a tolerância de 33 mV corresponde a ± 41 códigos, valor usado pelos casos automáticos, que conferem as leituras analógicas por faixa e não byte a byte.

3.8.3 Casos de Teste

Os Quadro 9 e 10 listam os casos de teste, com o procedimento, o resultado esperado e os requisitos que cada um comprova. Nos testes de integração, as medições feitas pela própria jiga são conferidas com instrumentos independentes – um multímetro digital para as tensões e um osciloscópio para os sinais PWM e os tempos –, pois não seria válido comprovar a jiga apenas com as suas próprias leituras. Nos testes de sistema, emprega-se um DUT de referência, isto é, um microcontrolador com um *firmware* escrito especificamente para o teste, de comportamento totalmente conhecido; assim, qualquer divergência pode ser atribuída à jiga, e não a um erro do DUT. O barramento I2C é verificado apenas no nível de sistema (CT20), porque a jiga dispõe de um único periférico I2C e não pode ser ligada a si mesma em *loopback*. A precisão temporal (CT19) também é verificada em *loopback*: uma rotina aciona uma saída digital ligada a uma entrada da própria jiga, e os instantes e as esperas registrados são

comparados com os medidos pelo osciloscópio.

Quadro 9 – Casos de teste de unidade e de integração com comandos diretos.

ID	Nível	Procedimento	Resultado esperado	Req.
CT01	Unidade	Calcular o CRC-16 da sequência ASCII `123456789`.	`0x29B1`.	RNF03
CT02	Unidade	Codificar e decodificar os quadros dos Quadros 20 e 21.	Bytes idênticos aos dos quadros; campos recuperados.	RF06
CT03	Unidade	Alterar um bit de um quadro válido.	Quadro descartado, sem resposta; o computador retransmite por tempo limite.	RNF03
CT04	Unidade	Inserir bytes espúrios antes do SOF e truncar um quadro.	Quadro incompleto descartado; ressincronização no SOF seguinte.	RNF03
CT05	Unidade	Transporte simulado descarta a primeira resposta.	Retransmissão com o mesmo SEQ; comando executado uma única vez.	RNF03
CT06	Unidade	Compilar a bateria da Figura 9 e um caso que não cabe em um quadro.	Rotina com os bytes esperados; caso grande recusado na validação.	RF07, RF10
CT07	Unidade	Executar baterias contra um `ClienteJiga` simulado com respostas conhecidas, incluindo rotinas concluídas, em erro e interrompidas.	Vereditos passou, falhou e inconclusivo conforme o previsto.	RF07, RF08
CT08	Integração	Enviar PING.	ACK com as versões do protocolo e do *firmware*.	RF06
CT09	Integração	Enviar um código inexistente e um parâmetro fora da faixa.	NACK com erros `0x01` e `0x02`; nenhuma saída alterada.	RF06
CT10	Integração	*Loopback* DAC1→A0 e DAC2→A1, varredura em 11 pontos de 124 a 3971 (faixa útil), com conferência por multímetro.	Erros dentro do Quadro 8 ($\pm$41 códigos), na leitura do ADC e na medição.	RF01, RF05
CT11	Integração	*Loopback* GPIO1–4→GPIO5–8, nas 16 combinações.	Máscara lida igual à escrita.	RF02, RF05
CT12	Integração	PWM nos três canais em frequências e razões cíclicas definidas, medidos com osciloscópio.	Dentro do Quadro 8.	RF03
CT13	Integração	*Loopback* MOSI→MISO (jiga mestra, modos 0 a 3 a 1 MHz e 239 bytes a 6,75 MHz) e UART4 TX→RX (9600, 115 200 e 1 000 000 *baud*).	Dados recebidos iguais aos enviados.	RF04, RF05

Fonte: elaborado pelo autor.

As ligações de *loopback* são feitas por *jumpers*: PA4→PA3 e PA5→PC0 no CT10; PC8–PC11→PC12, PD2, PG2 e PG3 no CT11; PB15→PC2 e PD1→PD0 no CT13; e PC8→PC12 no CT19. Os casos de integração são executados por um *script* do computador que envia os quadros pela USB e confere tanto as respostas quanto o

Quadro 10 – Casos de teste de rotinas, de sistema, de aceitação e de desempenho.

ID	Nível	Procedimento	Resultado esperado	Req.
CT14	Integração	Enviar uma rotina com um passo de código inexistente no meio.	NACK ROTINA_INVALIDA com o índice do passo; nenhum passo executado.	RF06, RF10
CT15	Integração	Durante uma rotina, enviar um comando direto e um novo RUN_ROUTINE; consultar o estado.	NACK OCUPADO para ambos; ROUTINE_STATUS atendido normalmente.	RF06, RNF03
CT16	Integração	Enviar ABORT durante uma espera longa.	Estado ABORTADA; saídas no estado seguro.	RF10, RNF05
CT17	Integração	Executar uma rotina cuja duração excede o tempo máximo informado.	Estado TEMPO_ESGOTADO; saídas no estado seguro.	RF10, RNF05
CT18	Integração	Retransmitir um RUN_ROUTINE com o mesmo SEQ durante a execução.	Resposta reenviada; rotina não reiniciada.	RNF03
CT19	Integração	Rotina em *loopback* GPIO1→GPIO5: acionar a saída, esperar tempos de 1 a 100 ms, medir a transição com ESPERA_GPIO; conferir com osciloscópio.	Instantes e esperas dentro do Quadro 8.	RF10
CT20	Sistema	DUT de referência (ESP32-S3), comandado por texto pela UART da jiga, lê e escreve a jiga escrava por I2C e por SPI e responde como escravo à jiga mestra.	Dados lidos e escritos iguais nos dois sentidos e nos dois barramentos.	RF04
CT21	Sistema	Executar dez vezes a mesma bateria, em *loopback* ou com um DUT de comportamento conhecido.	Mesmos estímulos, medições dentro das tolerâncias e mesmos vereditos; relatório gerado em cada execução.	RF08, RF09, RF10, RNF04
CT22	Aceitação	Executar a mesma bateria *black-box* sobre o *firmware* de comportamento conhecido “ventilador com histerese” no ESP32-S3 e no Arduino Uno.	Todos os vereditos coincidem com o comportamento esperado.	RF08, RNF02
CT23	Desempenho	Enviar 1.000 comandos PING consecutivos, medindo o tempo de ida e volta.	Latência máxima inferior a 50 ms; média e percentil 95 registrados.	RNF03
CT24	Desempenho	Enviar 10.000 comandos consecutivos, alternando escrita e leitura.	Nenhum erro não recuperado; taxa de retransmissão registrada.	RNF03
CT25	Sistema	A jiga emula o MPU-6500 e o BME280 no I2C, com valores fixos e alterados durante o caso; o DUT de referência os lê com *drivers* comuns e informa os valores pela UART.	Valores informados iguais aos simulados, dentro da tolerância.	RF11
CT26	Sistema	O BME280 emulado no SPI (modo registradores) e o MPU-6500 no I2C, ao mesmo tempo, lidos pelo DUT, inclusive durante uma rampa de temperatura.	Valores informados iguais aos simulados.	RF11, RF12
CT27	Integração	Ondas num registrador do sensor emulado, conferido em instantes conhecidos da rotina; senoide no DAC por DMA e triangular na	Valores e formas dentro do Quadro 8.	RF12

registro de depuração da jiga; os que não dependem de ligação rodam com a placa sozinha, e os de *loopback* são marcados à parte, para que a falta de um *jumper* não seja confundida com um defeito.

No CT20, o DUT de referência é um ESP32-S3 com um *firmware* escrito para o teste, que atende comandos de texto pela sua UART, ligada à UART4 da jiga: o computador envia o comando com UART_WRITE e lê a resposta com UART_READ, de modo que o caso inteiro roda pela própria jiga. O mesmo *firmware* executa, em uma tarefa própria e sem receber comando algum, o comportamento usado no CT22: lê a tensão de um "sensor" a cada 1 ms e liga uma saída acima de 1,80 V, desligando-a abaixo de 1,50 V. O Arduino Uno executa o mesmo comportamento, e a mesma bateria é aplicada aos dois. Os pontos de teste da histerese, 1,58 V e 1,72 V, ficam a 70 a 80 mV dos limiares e do ponto médio de 1,65 V, de modo que um *firmware* com um limiar único em 1,65 V falharia nos dois casos, e o erro dos conversores do DUT, de dezenas de milivolts, não altera o veredito.

Os casos CT25 e CT26 usam o mesmo DUT de referência do CT20, agora como cliente dos sensores emulados: ele lê o MPU-6500 e o BME280 com *drivers* escritos como os de um produto (*reset*, identificação, configuração e leitura, com a compensação da Bosch no BME280) e informa os valores em texto pela UART, que a bateria confere com tolerância. No CT27, os valores das ondas num sensor emulado são conferidos pela própria jiga, lendo o mapa da escrava em instantes conhecidos da rotina, o que dispensa ligações; as formas no DAC e no PWM são conferidas no osciloscópio.

3.8.4 Testes de Desempenho

Os testes de desempenho (CT23 e CT24) avaliam a comunicação sob uso contínuo, aspecto que os testes funcionais não cobrem, pois estes verificam apenas se cada comando produz o efeito correto. A latência de ida e volta é medida no computador, com um relógio de alta resolução, porque é esse o tempo que determina a duração de uma bateria e o risco de retransmissões indevidas. Além da média, registram-se o percentil 95 e o valor máximo, pois o escalonamento do sistema operacional do computador introduz atrasos ocasionais que a média esconderia, e são justamente esses atrasos que poderiam ultrapassar o tempo limite. O CT24 verifica se, ao longo de uma sequência longa de comandos, alguma falha escapa ao mecanismo de detecção e retransmissão (RNF03); a taxa de retransmissões é registrada como indicador da qualidade do enlace. Os atrasos da comunicação não afetam a temporização dentro de um caso de teste, que é executado na jiga e verificado separadamente pelo CT19.

4 RESULTADOS E DISCUSSÕES

Este capítulo apresenta a jiga construída e os resultados da verificação planejada na seção 3.8. A implementação foi conduzida em versões incrementais, cada uma fechada quando os casos de teste correspondentes passavam na placa; os resultados aqui relatados correspondem à versão 0.14.0 do *firmware* e da aplicação. Os casos que dependem da montagem em bancada com *jumpers*, instrumentos ou os DUTs ligados à jiga são identificados ao longo do texto e reunidos na seção 4.8.

4.1 SOLUÇÃO IMPLEMENTADA

O *firmware* implementa todos os comandos do Apêndice A: os 19 comandos diretos de sistema, DAC, GPIO, PWM, ADC, I2C, SPI e UART, os quatro comandos de rotina e os passos de espera, sobre a arquitetura em camadas da seção 3.5. A aplicação *desktop* implementa as classes da Figura 8: o cliente da jiga, com o controle de sequência e a retransmissão; o carregador de baterias YAML, que valida e compila os casos em rotinas; o motor de testes, que executa a bateria e emite os vereditos; o relatório, gravado em CSV e em JSON; e a janela principal. Uma mesma bateria pode ser executada pela janela ou pela linha de comando. Para a validação foram escritos ainda dois *firmwares* de DUT: o do ESP32-S3, que atende comandos de texto para o CT20 e executa o comportamento conhecido do CT22, e o do Arduino Uno, com o mesmo comportamento do CT22.

Toda a verificação é automatizada, como resume o Quadro 11. Os testes de unidade do *firmware* usam o *framework* Unity e rodam na própria placa; os módulos que não dependem do *hardware* (protocolo, execução, interpretador de rotinas e periféricos, com periféricos simulados) são também compilados e executados no computador, assim como os do *firmware* do DUT. Os testes da aplicação usam o *pytest*. Os testes de integração são executados por um *script* que envia os quadros à placa pela USB e confere tanto as respostas quanto o registro de depuração da jiga. Os testes que rodam no computador são executados também por integração contínua, a cada alteração enviada ao repositório.

4.2 TESTES DE UNIDADE

Os casos CT01 a CT07 passaram. O CRC-16 da sequência `123456789` resultou em `0x29B1` tanto no periférico de CRC do microcontrolador quanto na implementação em Python (CT01). Os 16 quadros dos Quadros 20 e 21 foram montados byte a byte e decodificados nos mesmos campos (CT02). A alteração de cada um dos 64 bits de um quadro de DAC_SET, fora do SOF, fez o quadro ser descartado

Quadro 11 – Testes automatizados e resultados na versão 0.14.0.

Conjunto	**Onde roda**	**Resultado**
Unidade do *firmware* (Unity)	placa (configuração de teste)	210 de 210
Unidade do *firmware*: execução, rotinas e ondas	computador e integração contínua	89 de 89
Unidade do *firmware* do DUT ESP32-S3	computador e integração contínua	22 de 22
Aplicação e casos de teste (*pytest*)	computador e integração contínua	625 de 625
Integração com a placa, sem ligações	placa e computador	73 de 73
Integração com os *jumpers* de *loopback*	placa e computador	74 de 74
Integração com o DUT de referência (CT20)	placa, computador e ESP32-S3	5 casos, executados em bancada

Fonte: elaborado pelo autor.

sem resposta e sem efeito na jiga simulada, e o cliente recuperou o comando por retransmissão (CT03). Bytes espúrios antes do SOF foram ignorados, e um quadro truncado foi descartado pelo tempo limite entre bytes, com a ressincronização no quadro seguinte (CT04). Com o transporte simulado descartando a primeira resposta, o cliente retransmitiu o comando com o mesmo SEQ, e o comando foi executado uma única vez (CT05); com três respostas perdidas seguidas, o cliente sinalizou a falha de comunicação, sem perda silenciosa. A bateria da Figura 9 foi compilada nos passos esperados, e um caso com 80 estímulos, que não cabe em um quadro, foi recusado na validação, antes de qualquer comunicação (CT06). Por fim, o motor de testes, executado contra a jiga simulada, emitiu os vereditos previstos para rotinas concluídas com medições certas e erradas, rotinas finalizadas em ERRO, rotinas que esgotaram o tempo máximo, interrupção pelo usuário e perda de comunicação (CT07).

A jiga simulada usada nesses testes espelha a lógica do *firmware*, e os dois são conferidos pelos mesmos vetores de teste. Isso permitiu desenvolver e testar a aplicação sem a placa, e cada caso de integração usa a jiga simulada para definir a resposta esperada antes de ser enviado à placa.

4.3 TESTES DE INTEGRAÇÃO COM COMANDOS DIRETOS

O PING foi respondido com as versões do protocolo e do *firmware* (CT08), e os comandos inexistentes e com parâmetros fora da faixa receberam os NACKs previstos, sem alterar nenhuma saída (CT09). Entre os 68 casos de integração sem ligações, há casos para cada comando direto, com parâmetros válidos e inválidos, para o comportamento do receptor diante de quadros corrompidos, truncados e com bytes espúrios, para a retransmissão e para a resposta OCUPADO.

4.3.1 DAC e ADC

A Tabela 2 apresenta a tensão medida com multímetro nos dois canais do DAC, sem carga, com a tensão do pino de 3,3 V da placa medida em 3,300 V. No meio da escala, o erro foi de +3 mV e -2 mV, cerca de 0,1%. Nas pontas, a saída satura a cerca de 50 mV de cada trilho, comportamento do *buffer* de saída que motivou a definição da faixa útil (subseção 3.4.4): os códigos 0 e 1 produzem a mesma tensão, e o código 4095 fica 46 mV abaixo do ideal.

Tabela 2 – Tensão de saída do DAC medida com multímetro, sem carga.

Código	Ideal (V)	PA4, canal 1 (V)	PA5, canal 2 (V)	Erro (mV)
0	0,000	0,052	0,051	+52 / +51
1	0,0008	0,052	0,051	+51 / +50
2048	1,650	1,653	1,648	+3 / -2
4095	3,300	3,254	3,254	-46 / -46

Fonte: elaborado pelo autor.

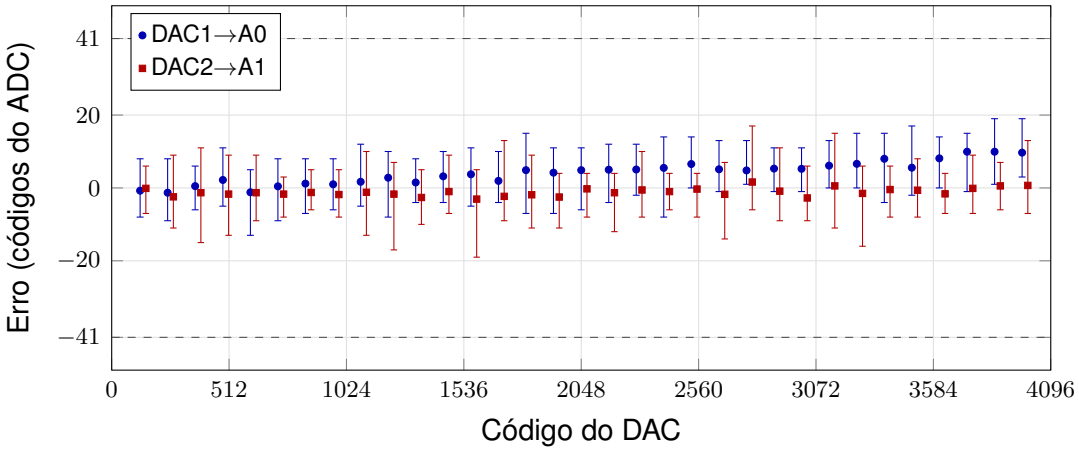
No *loopback* DAC$\rightarrow$ADC (CT10), cada canal do DAC foi ligado a uma entrada analógica e varrido em 33 pontos de 124 a 3971, a faixa útil, com 16 leituras do ADC por ponto. A Figura 11 mostra o erro de cada ponto, e a Tabela 3 resume os dois canais. Nenhuma das 1.056 leituras ficou fora da tolerância de ± 41 códigos (33 mV); o pior caso isolado foi de 19 códigos (15,3 mV), menos da metade da tolerância. O erro sistemático é pequeno: um ganho de +0,25% no canal 1, que chega a cerca de +10 códigos (8 mV) no topo da faixa, e de +0,04% no canal 2, com desvio de 1 a 2 códigos nos dois. A comparação com a Tabela 2 indica que a diferença entre os canais vem do DAC, e não do ADC: no código 2048, o multímetro mediu +3,7 e $-2{,}5$ códigos nos dois canais, e o *loopback*, +4,9 e $-0{,}2$. O que define a pior leitura isolada é o ruído, com faixa de 15 a 30 códigos entre o mínimo e o máximo das 16 leituras de um ponto, e não o erro médio. O critério de 1% do fundo de escala é, portanto, atendido com margem de cerca de duas vezes; se uma aplicação exigir tolerância menor, o caminho é a média de várias conversões no *firmware*, sem alteração do *hardware*. O caso automático do CT10, com os 11 pontos do Quadro 9, e o CT11, com as 16 combinações de saídas e entradas digitais, também passaram com os *jumpers*.

Tabela 3 – Resumo do *loopback* DAC$\rightarrow$ADC na faixa útil (33 pontos, 16 leituras por ponto).

Canal	Reta ajustada (leitura)	Erro médio por ponto	Pior leitura	Fora de ± 41
DAC1$\rightarrow$A0	$1{,}00253 \cdot \text{DAC} - 1{,}00$	$-1{,}3$ a +9,9 códigos	+19 códigos (+15,3 mV)	0 de 528
DAC2$\rightarrow$A1	$1{,}00042 \cdot \text{DAC} - 1{,}99$	$-3{,}1$ a +1,6 códigos	-19 códigos ($-15{,}3$ mV)	0 de 528

Fonte: elaborado pelo autor.

Figura 11 – Erro da leitura do ADC no *loopback* DAC→ADC (média e faixa de 16 leituras por ponto).



Fonte: elaborado pelo autor. As linhas tracejadas marcam a tolerância de ±41 códigos (33 mV); os pontos do canal 2 foram deslocados 25 códigos para a direita, para não se sobreporem.

4.3.2 PWM

A Tabela 4 apresenta as medições do CT12 com osciloscópio, em seis combinações que cobrem os três canais, os extremos da faixa de frequência (1 Hz e 100 kHz) e razões cíclicas de 12,34% a 90%. Todas ficaram dentro do critério. A coluna “esperado” mostra o valor que os registradores do temporizador efetivamente produzem, calculado pela aplicação: em 20 kHz no TIM2, 12,34% resulta em 12,33%, pois o período tem 5400 contagens; em 50 Hz no TIM3, de 16 bits, o divisor necessário dá 49,9997 Hz. Esses arredondamentos são duas ordens de grandeza menores que a tolerância. A resolução de leitura do osciloscópio, de ±10 Hz em 20 kHz e 100 kHz, é inferior ao critério de 1% (200 Hz e 1 kHz).

Tabela 4 – PWM medido com osciloscópio (CT12).

		Pedido		**Esperado**		**Medido**	
Canal	**Pino**	**Hz**	**%**	**Hz**	**%**	**Hz**	**%**
PWM1 (TIM1)	PE9	1000	50,00	1000,0000	50,00	1000	50,00
PWM1 (TIM1)	PE9	100 000	90,00	100 000,0000	90,00	100 000	90,00
PWM2 (TIM2)	PA0	20 000	12,34	20 000,0000	12,33	20 000	12,34
PWM2 (TIM2)	PA0	1	50,00	1,0000	50,00	1	50,00
PWM3 (TIM3)	PA6	50	75,00	49,9997	75,00	50	75,00
PWM3 (TIM3)	PA6	100 000	25,00	100 000,0000	25,00	100 000	25,00

Fonte: elaborado pelo autor.

4.3.3 I2C, SPI e UART

Sem ligações, os casos de integração verificaram as configurações, a carga e a leitura do mapa do escravo I2C e dos *buffers* do SPI e da UART, as validações de parâmetros, as respostas NAO_CONFIGURADO e, no papel de mestre sem dispositivo no barramento I2C, a falta de reconhecimento, informada como ERRO_PERIFERICO sem travar a jiga. No CT13, todos os comandos de SPI e de UART foram aceitos e executados na placa; a conferência dos dados recebidos depende dos *jumpers* de *loopback* (seção 4.8).

4.4 ROTINAS EXECUTADAS NA JIGA

Os casos CT14 a CT18 passaram na placa. Uma rotina com um passo inválido foi recusada com NACK ROTINA_INVALIDA e o índice do passo, sem executar nenhum dos anteriores (CT14). Durante uma rotina, um comando direto e um novo RUN_ROUTINE receberam OCUPADO, enquanto ROUTINE_STATUS e PING continuaram atendidos (CT15). Um ABORT enviado no meio de uma espera de 10 s interrompeu a rotina de imediato, com a jiga no estado seguro (CT16), e uma rotina mais longa que o tempo máximo foi interrompida no estado TEMPO_ESGOTADO (CT17). Um RUN_ROUTINE retransmitido com o mesmo SEQ recebeu a resposta guardada, sem reiniciar a rotina (CT18).

Para a base de tempo (CT19), uma rotina sem ligações executou ESPERAs de 1, 10 e 100 ms entre duas leituras das entradas digitais. Os intervalos registrados pela jiga entre as duas leituras foram de 1028, 10 029 e 100 029 µs: cerca de 28 µs acima da espera pedida, de modo constante, valor que corresponde à execução da leitura e ao retorno da notificação que acorda a `TestExecTask`. Esse custo fixo por passo fica dentro da tolerância temporal de 50 µs do Quadro 8 e é cerca de 35 vezes menor que a resolução de 1 ms do relógio do sistema operacional, que seria a precisão obtida sem a base de tempo dedicada. A conferência dos instantes com o osciloscópio e o tempo de reação medido pelo ESPERA_GPIO, com a saída GPIO1 ligada à entrada GPIO5, são feitos em bancada.

4.5 TESTES DE SISTEMA E DE ACEITAÇÃO

O CT20, com o ESP32-S3 como DUT de referência, foi verificado contra a jiga simulada ligada a um modelo do DUT, conferido pelos mesmos vetores dos testes de unidade do *firmware* do ESP32-S3; o *firmware* do DUT foi gravado e testado sozinho, pela sua porta USB, em todos os comandos. Sem o DUT ligado, cada um dos cinco casos falha apenas na resposta do DUT, sem travar a jiga.

Para o CT21, a bateria que roda na jiga sem ligações (mapa do escravo

I2C, *buffers* vazios do SPI e da UART, estímulos e esperas) foi executada dez vezes seguidas pela linha de comando: os três casos passaram nas dez execuções, com os mesmos vereditos e as mesmas medições, nenhuma retransmissão em 140 comandos e um relatório gerado em cada execução. A bateria de *loopback* completa, com os *jumpers*, e o CT22, com os dois DUTs, são executados em bancada. Para o CT22, a mesma bateria foi executada contra modelos simulados do DUT: o comportamento conhecido passou nos cinco casos, um *firmware* sem histerese falhou exatamente nos dois casos da histerese e um DUT parado falhou nos dois casos que esperam a saída ligada, o que mostra que a bateria distingue os comportamentos.

4.6 DESEMPENHO DA COMUNICAÇÃO

A Tabela 5 e a Figura 12 apresentam os tempos de ida e volta medidos no computador (Windows 11, Python 3.14), com a jiga ligada à porta USB OTG. O CT23 e o CT24 passaram: o maior tempo entre os 11 000 comandos foi de 3,1 ms, cerca de 16 vezes menor que o limite de 50 ms e 30 vezes menor que o tempo limite de resposta de 100 ms, e não houve nenhum erro não recuperado nem retransmissão. Uma segunda execução, com a versão 0.11.0, repetiu os valores com diferenças de centésimos de milissegundo.

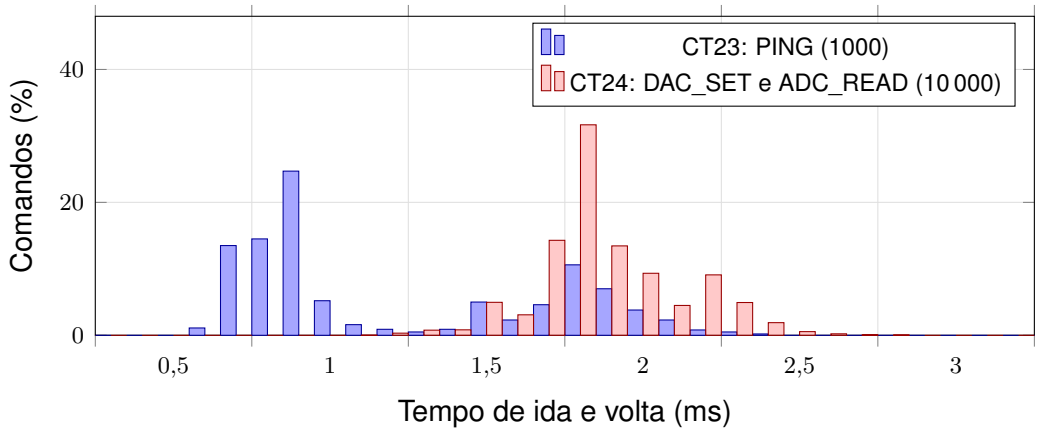
Tabela 5 – Tempo de ida e volta dos comandos (CT23 e CT24).

		Tempo de ida e volta (ms)						
Caso	**Comandos**	**Média**	**Mediana**	**P95**	**P99**	**Máx.**	**Erros**	**Retr.**
CT23	1000	1,46	1,17	2,26	2,47	2,67	0	0
CT24	10 000	2,13	2,07	2,54	2,69	3,08	0	0

Fonte: elaborado pelo autor. CT23: PING; CT24: DAC_SET e ADC_READ alternados. Erros: não recuperados; Retr.: retransmissões.

Dois aspectos merecem destaque. O primeiro é a diferença entre os dois casos: o PING é respondido pela própria `CommTask`, enquanto os comandos do CT24 passam pela fila e pela `TestExecTask`, o que acrescenta, em média, cerca de 0,7 ms. Esse acréscimo inclui a troca entre as duas tarefas e a execução do comando, e é pequeno porque a `TestExecTask`, de prioridade maior, executa o comando assim que ele entra na fila (seção 3.5). O segundo é a cauda da distribuição: em uma primeira execução do CT24, um único comando levou 15 ms, efeito do escalonamento do sistema operacional do computador, que a média e o percentil 99 não revelam e que justifica o registro do máximo. Mesmo esse valor isolado ficou bem abaixo do tempo limite. A ausência de retransmissões em 11 000 comandos indica que, na ligação USB direta, o mecanismo de retransmissão quase nunca é acionado; ele permanece

Figura 12 – Distribuição do tempo de ida e volta dos comandos (intervalos de 0,1 ms).



Fonte: elaborado pelo autor.

necessário para os casos de perda de alinhamento e de reinício de um dos lados, verificados no CT03 a CT05.

Os tempos medidos também confirmam a escolha de executar as rotinas na jiga (subseção 3.6.3). Comandado pelo computador, o intervalo entre um estímulo e uma leitura variaria entre cerca de 1,4 e 3 ms, com casos isolados acima de 15 ms; executado em uma rotina, o mesmo intervalo tem um custo fixo de cerca de 28 µs por passo, cem vezes menor, e é registrado com resolução de 1 µs.

4.7 EMULAÇÃO DE SENSORES E GERADOR DE ONDAS

A conversão do BME280 foi conferida contra a folha de dados: com os coeficientes do exemplo do fabricante, os valores brutos do exemplo resultaram em 25,08 °C e 100 653,27 Pa, iguais aos publicados, tanto na aplicação quanto no *driver* do DUT de referência, testado no computador. A inversa acertou a temperatura, a pressão e a umidade com erro de até 0,005 em toda a faixa do sensor. Com o DUT simulado, que lê a jiga com os mesmos *drivers* do ESP32-S3, as baterias do CT25 e do CT26 passaram em todos os casos, inclusive com o BME280 no SPI e o MPU-6500 no I2C ao mesmo tempo e durante uma rampa de temperatura, em que a pressão e a umidade lidas se mantiveram dentro de 0,05 graças às ondas de correção.

Na placa, sem ligações, uma bateria com ondas num registrador do MPU-6500 emulado conferiu os valores lendo o mapa da escrava em instantes conhecidos da rotina: uma rampa de 0 a 1 g em 1 s valia exatamente 0,25 g, 0,5 g e 1 g (valor final mantido) a 250 ms, 500 ms e 1 s, e uma onda quadrada de 200 ms alternou entre 35 °C e 25 °C nos instantes previstos, em seis execuções seguidas. A mesma bateria executou uma senoide de 1 kHz no DAC por DMA, com passo de 10 µs, e uma onda triangular na razão do PWM; as rotinas concluíram e a jiga continuou respondendo normalmente, o que confirma a parada das ondas no estado seguro. As formas no

DAC e no PWM e a leitura dos sensores pelo ESP32-S3 são conferidas em bancada.

4.8 CASOS EXECUTADOS EM BANCADA

Com os *jumpers* de *loopback*, a suíte de integração passou em todos os 74 casos, incluindo o CT10, o CT11, o CT13 e o CT19 com a ligação GPIO1→GPIO5, e a bateria de *loopback* do CT21 passou nas dez execuções, com os mesmos vereditos e nenhuma retransmissão em 240 comandos. Essa etapa revelou um defeito que os testes sem ligações não podiam mostrar: na primeira transferência do SPI como mestre depois de a jiga ter sido escrava, os dados chegavam atrasados em 4 bytes. A causa era a FIFO de recepção do SPI2, que guarda até 4 bytes e não é esvaziada quando o periférico é desligado pelo HAL; os bytes recebidos como escrava ficavam na frente dos novos. A correção, reiniciar o periférico pelo controlador de *reset* ao desligá-lo, foi validada na mesma montagem. Faltam, em bancada:

- CT19: conferência dos instantes com o osciloscópio;
- CT20: ESP32-S3 ligado aos barramentos I2C e SPI e à UART4 da jiga;
- CT22: bateria do ventilador com histerese no ESP32-S3 e no Arduino Uno;
- CT25 e CT26: sensores emulados lidos pelo ESP32-S3 no I2C e no SPI;
- CT27: formas das ondas no DAC e no PWM no osciloscópio.

4.9 CUSTO

O custo da jiga é dado pela lista de materiais da Tabela 1, composta por uma placa de desenvolvimento e componentes passivos de uso comum, sem placa dedicada nem instrumentos próprios. A comparação com as plataformas comerciais de HIL é apresentada quando a cotação dos itens for concluída.

4.10 ATENDIMENTO AOS REQUISITOS

O Quadro 12 relaciona cada requisito aos resultados obtidos.

Entre as limitações observadas, três decorrem do *hardware* e estão documentadas. A primeira é a faixa útil do DAC, de 0,1 a 3,2 V, consequência do *buffer* de saída, cuja manutenção foi uma escolha para preservar a tensão sob carga. A segunda é a ausência de resistores internos nas entradas digitais, imposta pela tolerância a 5 V: uma entrada solta lê um valor indefinido, e um DUT com saída de dreno aberto exige um resistor externo. A terceira é a existência de um único periférico I2C, que impede verificar o barramento em *loopback* e torna o CT20 a única verificação dos dados do I2C. Do lado da aplicação, a verificação de tempo de reação depende de o estímulo

Quadro 12 – Atendimento aos requisitos.

Req.	**Evidência**	**Situação**
RF01	DAC medido (Tabela 2) e CT10	atendido na faixa útil de 0,1 a 3,2 V
RF02	CT11, 16 combinações	atendido
RF03	CT12 (Tabela 4)	atendido
RF04	casos sem ligação; CT13 com *jumpers*; CT20 em bancada	atendido no *loopback*; DUT em bancada
RF05	CT10, CT11; UART no CT13	atendido
RF06	CT02, CT08, CT09 e os 68 casos de integração	atendido
RF07	CT06; baterias YAML versionadas com o projeto	atendido
RF08	CT07 e CT22 simulado	atendido; DUTs em bancada
RF09	relatórios CSV e JSON em cada execução (CT21)	atendido
RF10	CT14 a CT19	atendido; osciloscópio em bancada
RF11	CT25 e CT26 com o DUT simulado	atendido; ESP32-S3 em bancada
RF12	CT27 na placa (valores nos instantes previstos)	atendido; osciloscópio em bancada
RNF01	lista de materiais	cotação em andamento
RNF02	CT22	em bancada
RNF03	CT03 a CT05, CT18, CT23 e CT24	atendido
RNF04	CT21 sem ligações e com o *loopback* (dez execuções idênticas cada)	atendido
RNF05	estado seguro (CT16, CT17), pinos FT (subseção 3.4.3)	atendido
RNF06	Apêndice A; módulos testados isoladamente	atendido

Fonte: elaborado pelo autor.

ser aplicado no passo imediatamente anterior ao ESPERA_GPIO, o que é garantido pela compilação das baterias.

5 CONSIDERAÇÕES FINAIS E SUGESTÕES PARA TRABALHOS FUTUROS

Este trabalho desenvolveu uma jiga de testes *hardware-in-the-loop* de baixo custo, baseada na placa NUCLEO-F767ZI, capaz de gerar estímulos analógicos, digitais e PWM, de emular e comandar dispositivos nos barramentos I2C, SPI e UART e de adquirir as respostas de um microcontrolador-alvo, com as rotinas de teste descritas em arquivos de texto e executadas automaticamente por uma aplicação *desktop*. Os objetivos específicos foram atendidos da seguinte forma.

A revisão das soluções acadêmicas e comerciais (Capítulo 2) fundamentou as principais decisões de projeto: a abordagem *black-box*, a separação em três camadas e a escolha de um microcontrolador que reúne em um só *chip* todos os periféricos necessários. O *firmware* implementa o protocolo completo do Apêndice A, com 19 comandos diretos, os comandos de rotina e uma base de tempo de 1 µs, sobre uma arquitetura em camadas que permitiu testar no computador, sem a placa, os módulos de protocolo, de execução e de periférico. A aplicação *desktop* permite descrever baterias de testes em YAML, com as grandezas em unidades físicas, executá-las pela interface gráfica ou pela linha de comando e gravar um relatório de cada execução. A leitura das respostas e o veredito automático foram implementados com três resultados possíveis – passou, falhou e inconclusivo –, o que impede que uma falha de comunicação ou uma rotina interrompida seja atribuída ao DUT. A validação com dois microcontroladores distintos foi preparada com um comportamento conhecido implementado no ESP32-S3 e no Arduino Uno e com uma bateria comum aos dois, cuja capacidade de distinguir comportamentos foi verificada com modelos simulados; a execução em bancada completa essa etapa.

Ao longo do desenvolvimento, a jiga ganhou dois recursos que ampliam o alcance dos testes sem mudar a sua arquitetura. O primeiro é a emulação de sensores comerciais, o MPU-6500 e o BME280, que o DUT lê com os seus próprios *drivers*; ela foi construída sobre o mapa de registradores que já existia, com uma máscara de bits graváveis, um segundo endereço e um modo registradores na escrava SPI, enquanto os modelos dos sensores, inclusive a compensação inversa do BME280, ficaram na aplicação. O segundo é o gerador de ondas, que faz a própria jiga variar estímulos e grandezas no tempo durante a rotina, com qualquer forma descrita por uma tabela de pontos e, no DAC, com amostras a cada microssegundo por DMA. A emulação no SPI tem uma restrição inerente ao barramento, a pausa que o mestre precisa deixar depois do endereço, que foi documentada.

Os resultados medidos confirmam as escolhas de projeto. Os conversores atenderam ao critério de 1% do fundo de escala na faixa útil, com o pior caso em menos da metade da tolerância, e o PWM ficou dentro do critério de 1 Hz a 100 kHz. A

execução das rotinas na própria jiga, em vez de comandos isolados enviados pelo computador, reduziu o intervalo entre um estímulo e uma leitura de alguns milissegundos, com variações imprevisíveis, para um custo fixo de cerca de 28 µs por passo, medido com resolução de 1 µs. A comunicação mostrou folga ampla: em 11 000 comandos, o maior tempo de ida e volta foi de 3,1 ms, cerca de trinta vezes abaixo do tempo limite, sem nenhum erro nem retransmissão. A escolha de validar a rotina inteira antes de executá-la, de aplicar o estado seguro ao fim de cada rotina e de tornar segura a retransmissão, pelo número de sequência, fez com que nenhum dos casos de falha provocados nos testes deixasse um estímulo aplicado ou um comando executado duas vezes.

O processo de desenvolvimento também merece registro. Cada versão foi fechada apenas quando os casos de teste correspondentes passavam na placa, e os casos de integração foram escritos contra uma jiga simulada que espelha o *firmware*, conferida pelos mesmos vetores de teste. Essa disciplina permitiu evoluir o protocolo e a aplicação com segurança e deixou mais de oitocentos testes automatizados que podem ser repetidos a cada alteração, o que é, em si, uma aplicação da prática de teste que a jiga pretende facilitar.

Entre as limitações, destacam-se a faixa útil do DAC, de 0,1 a 3,2 V, consequência do *buffer* de saída; as entradas digitais sem resistor interno, imposição da tolerância a 5 V; e a existência de um único periférico I2C, que impede verificar esse barramento em *loopback*. Nenhuma delas compromete o uso previsto, e todas estão documentadas nas restrições de uso.

5.1 SUGESTÕES PARA TRABALHOS FUTUROS

A partir dos resultados e das limitações observadas, sugerem-se as seguintes continuações:

- **placa de interface dedicada**, com conectores identificados, resistores de *pull-up* e *pull-down* selecionáveis nas entradas digitais, divisores de tensão para sinais analógicos de 5 V e proteção contra ligação incorreta, substituindo os *jumpers* e reduzindo os erros de montagem;
- **mais modelos de sensores**, como magnetômetros, sensores de distância e conversores analógico-digitais externos, aproveitando o mapa, as máscaras e o gerador de ondas que já existem;
- **SPI sem a pausa depois do endereço**, com uma pequena lógica programável que entregue o registrador pedido dentro do próprio byte, para atender mestres que usam DMA;

- **média de conversões no ADC**, configurável por verificação, para reduzir o ruído que hoje define a pior leitura isolada e permitir tolerâncias menores;
- **captura de sinais PWM do DUT**, medindo frequência e razão cíclica com os temporizadores da jiga, o que ampliaria as verificações possíveis sem osciloscópio;
- **um segundo periférico I2C**, em outra placa ou microcontrolador, para permitir o *loopback* do barramento e emular dois dispositivos ao mesmo tempo;
- **integração com ferramentas de integração contínua**, executando as baterias na jiga a cada alteração do *firmware* do DUT e publicando os relatórios, já possível pela linha de comando e pelos códigos de saída da aplicação;
- **editor gráfico de baterias** na aplicação, para que o usuário monte os casos sem editar o arquivo YAML;
- **portabilidade da jiga** para outros microcontroladores, reescrevendo apenas a camada de abstração de *hardware*, como previsto na arquitetura do *firmware*.

REFERÊNCIAS

Altium Resources. **Como Projetar para Teste em Sistemas Embarcados**. 2024. Altium. Disponível em: <https://resources.altium.com/pt/p/how-design-test-embedded-systems>. Citado 3 vezes nas páginas 18, 24 e 49.

BARBOSA, J. R. **Estudo e Definição de uma Metodologia de Teste de Software no Contexto de Sistemas Embarcados Críticos**. s.d. Disponível em: <https://ww2.inf.ufg.br/ppgcc/sites/www.inf.ufg.br.mestrado/files/uploads/Dissertacoes/JacsonRodrigues.pdf>. Citado na página 19.

BENOVICS, T. *et al.* Remote IoT education laboratory for microcontrollers based on the STM32 chips. **Sensors**, v. 22, n. 4, 2022. PMC8880355. Disponível em: <https://www.mdpi.com/1424-8220/22/4/1440>. Citado 5 vezes nas páginas 15, 19, 20, 21 e 25.

BRANAM, S. **Unit Testing for Embedded Software Development**. 2025. Dojo Five Blog. Disponível em: <https://dojofive.com/blog/unit-testing-for-embedded-software-development/>. Citado 4 vezes nas páginas 14, 19, 31 e 48.

CABRAL, T. F. **Sistema de Detecção de Anomalias em Sistemas Embarcados**. 2020. Trabalho de Conclusão de Curso. Disponível em: <https://www.ecomp.poli.br/ListaTCC/20201/Tiago%20Farias%20Cabral.pdf>. Citado na página 19.

dSPACE. **HIL Simulator Hardware**. s.d. Disponível em: <https://www.dspace.com/en/pub/home/products/hw/simulator_hardware/scalexio.cfm>. Citado 4 vezes nas páginas 13, 14, 15 e 20.

Engenharia Híbrida. **Jiga de Teste Automatizada com Software Integrado**. s.d. Ecossistema Hub. Disponível em: <https://www.engenhariahibrida.com.br/jiga-de-testes-automatizados>. Citado na página 21.

Fidus Systems. **Mastering Embedded Software Testing: A Complete Guide to Tools and Techniques**. 2025. Fidus Blog. Disponível em: <https://fidus.com/blog/mastering-embedded-software-testing-a-complete-guide-to-tools-and-techniques/>. Citado na página 18.

GIL, A. C. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017. Citado na página 23.

LINS, F. C. *et al.* **Estudo de Metodologia de Desenvolvimento Baseado em Testes Aplicado a Sistemas Embarcados de Tempo Real**. 2016. Disponível em: <https://dspace.sti.ufcg.edu.br/handle/riufcg/18630>. Citado 4 vezes nas páginas 14, 19, 31 e 48.

MENDONçA, R. N. **Metodologia de Teste de Sistemas Embarcados Aplicada a Medidores Inteligentes e Sistemas de Medição Centralizada**. 2019. Disponível em: <https://repositorio.ufc.br/bitstream/riufc/45316/3/2019_tcc_rnmendonca.pdf>. Citado na página 19.

Microchip Technology. **ATmega328P: 8-bit AVR microcontroller with 32K bytes in-system programmable flash**. s.d. Folha de dados. Características elétricas: tensão mínima de entrada em nível alto. Disponível em: <https://www.microchip.com/en-us/product/atmega328p>. Citado na página 28.

National Instruments. **TestStand and LabVIEW HIL Test Systems**. s.d. Disponível em: <https://ni.nubicom.co.kr/img/pdf/HIL_test_system.pdf>. Citado 3 vezes nas páginas 14, 15 e 20.

Portal Embarcados. **Teste de software em sistemas embarcados automotivos**. 2020a. Embarcados. Disponível em: <https://embarcados.com.br/teste-de-software-em-sistemas-embarcados-automotivos/>. Citado 2 vezes nas páginas 13 e 18.

Portal Embarcados. **Testes automatizados: como faço com o hardware?** 2020b. Embarcados. Disponível em: <https://embarcados.com.br/testes-automatizados-e-o-hardware/>. Citado 4 vezes nas páginas 14, 19, 24 e 49.

PRODANOV, C. C.; FREITAS, E. C. de. **Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico**. 2. ed. Novo Hamburgo: Feevale, 2013. Citado na página 23.

QU, B.; LIN, H. Design of acceleration sensor simulation system based on STM32. **Advanced Materials Research**, Trans Tech Publications, v. 588-589, p. 940–943, 2012. Disponível em: <https://www.researchgate.net/publication/288487563_Design_of_Acceleration_Sensor_Simulation_System_Based_on_STM32>. Citado 2 vezes nas páginas 21 e 25.

STMicroelectronics. **STM32F7 Series – ARM Cortex-M7 MCUs**. s.d. Datasheet e Referência Técnica. Disponível em: <https://www.st.com/en/microcontrollers-microprocessors/stm32f7-series.html>. Citado 2 vezes nas páginas 18 e 25.

STMicroelectronics. **STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: datasheet DS11532**. s.d. Folha de dados. Tabela de características de tensão dos pinos de entrada e saída (pinos FT). Disponível em: <https://www.st.com/resource/en/datasheet/stm32f767zi.pdf>. Citado na página 27.

TheEmbeddedKit. **Alternatives to Robot Framework for Embedded Testing**. 2024. The Embedded Kit Blog. Disponível em: <https://theembeddedkit.io/blog/alternative-to-robot-framework/>. Citado 2 vezes nas páginas 21 e 43.

Apêndices

APÊNDICE A – ESPECIFICAÇÃO COMPLETA DO PROTOCOLO DE COMUNICAÇÃO

Este apêndice especifica o protocolo de comandos entre a aplicação *desktop* (computador) e o *firmware* da jiga, transportado sobre USB-CDC. O resumo do protocolo e as justificativas de projeto estão na seção 3.6.

A.1 PRINCÍPIOS E CONVENÇÕES

O protocolo segue o modelo requisição–resposta: o computador é o único a iniciar a comunicação, e cada comando enviado recebe exatamente uma resposta da jiga, um ACK (confirmação, com dados quando aplicável) ou um NACK (recusa, com o código do erro). O computador só envia um novo comando depois de receber a resposta do anterior.

Há dois tipos de comando. Os **comandos diretos** (Apêndice A.5) executam uma única operação sobre um periférico e respondem ao seu término; servem à operação manual e aos testes de integração. Os **comandos de rotina** (Apêndice A.6) transferem à jiga uma rotina de teste completa – uma sequência de estímulos, esperas e aquisições –, que a jiga executa sozinha; o computador acompanha a execução por consultas e, ao final, lê as medições registradas.

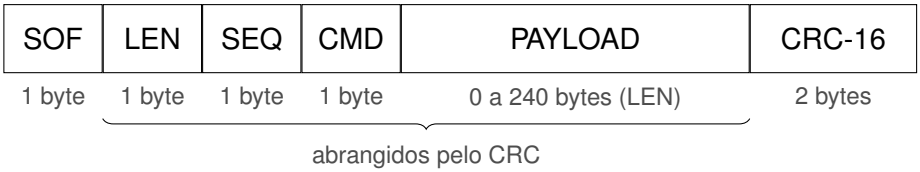
As convenções adotadas são:

- campos de vários bytes são transmitidos em ordem *little-endian*, isto é, o byte menos significativo primeiro;
- os tipos são indicados por `u8`, `u16` e `u32`, para inteiros sem sinal de 8, 16 e 32 bits;
- os canais de DAC são numerados de 1 a 2, os de PWM de 1 a 3, os de ADC de 0 a 2 (A0 a A2) e as saídas digitais GPIO1 a GPIO4 por 1 a 4;
- as entradas digitais GPIO5 a GPIO8 são lidas em conjunto, na forma de uma máscara de 8 bits, em que o bit 0 corresponde ao GPIO5 e o bit 3 ao GPIO8; os bits 4 a 7 são sempre zero;
- as numerações de pinos e canais correspondem ao mapeamento dos Quadros 3 e 4.

A.2 FORMATO DO QUADRO

Como o USB-CDC entrega um fluxo contínuo de bytes, sem fronteiras entre mensagens, cada mensagem é encapsulada em um quadro com o formato da Figura 13 e do Quadro 13.

Figura 13 – Formato do quadro do protocolo PC–Jiga.



Fonte: elaborado pelo autor.

Quadro 13 – Campos do quadro do protocolo.

Campo	**Tamanho**	**Descrição**
SOF	1 byte	Início de quadro, sempre `0x7E`.
LEN	1 byte	Número de bytes do campo PAYLOAD (0 a 240).
SEQ	1 byte	Número de sequência. O computador o incrementa a cada novo comando (volta a 0 após 255), e a jiga o repete na resposta.
CMD	1 byte	Código do comando (Quadros 15 a 17). Na resposta de sucesso, é o código do comando com o bit 7 ligado; na recusa, é `0xFF`.
PAYLOAD	0 a 240 bytes	Parâmetros do comando ou dados da resposta.
CRC-16	2 bytes	Verificação de integridade calculada sobre LEN, SEQ, CMD e PAYLOAD (Apêndice A.3).

Fonte: elaborado pelo autor.

O tamanho máximo de um quadro é de 246 bytes. O receptor sincroniza-se procurando o byte SOF e valida o quadro pelo CRC. Se o CRC falhar, ou se o LEN for maior que 240, o receptor descarta o quadro, **sem responder**, e volta a procurar o próximo SOF; o mesmo acontece com um quadro incompleto, descartado quando o intervalo entre dois bytes passa de 20 ms. A jiga não responde a um quadro corrompido porque, nele, nem o SEQ nem o CMD são confiáveis: uma resposta com um SEQ errado poderia ser associada pelo computador a outro comando. A recuperação fica a cargo do computador, que retransmite o comando ao esgotar o tempo limite de resposta (Apêndice A.7).

A.3 VERIFICAÇÃO DE INTEGRIDADE

O CRC é o CRC-16/CCITT-FALSE: polinômio `0x1021`, valor inicial `0xFFFF`, sem inversão de bits na entrada ou na saída e sem operação final de XOR. O resultado de 16 bits é transmitido em *little-endian*. Uma implementação correta produz `0x29B1` para a sequência ASCII `123456789`, valor que pode ser usado como teste de unidade.

A.4 RESPOSTAS

A.4.1 ACK

Quando o comando é aceito, a jiga responde com o mesmo SEQ e com CMD igual ao código do comando com o bit 7 ligado (`CMD OU 0x80`). O campo PAYLOAD carrega os dados de retorno, quando existirem. Para os comandos diretos, o ACK indica que a operação foi executada; para RUN_ROUTINE, indica que a rotina foi validada e sua execução começou.

A.4.2 NACK

Quando o comando não pode ser executado, a jiga responde com o mesmo SEQ, CMD igual a `0xFF` e PAYLOAD com o código do comando recusado (`u8`) e o código do erro (`u8`), conforme o Quadro 14. No erro ROTINA_INVALIDA, acrescenta-se um terceiro byte com o índice do passo recusado (a partir de 0), ou `0xFF` quando o problema está no cabeçalho da rotina (tempo máximo ou número de passos fora da faixa) ou em bytes que sobram depois do último passo. Nenhuma ação é executada quando há NACK.

Quadro 14 – Códigos de erro do NACK.

Código	Nome	Significado
`0x01`	CMD_DESCONHECIDO	O código do comando não existe.
`0x02`	PARAMETRO_INVALIDO	Parâmetro fora da faixa permitida (por exemplo, canal inexistente ou valor do DAC acima de 4095).
`0x03`	NAO_CONFIGURADO	O periférico exige configuração prévia (I2C, SPI ou UART).
`0x04`	CRC_INVALIDO	Reservado; não é enviado. Um quadro com CRC inválido é descartado sem resposta, pois o SEQ e o CMD recebidos não são confiáveis.
`0x05`	TAMANHO_EXCEDIDO	Reservado; não é enviado. Um LEN maior que 240 também descarta o quadro sem resposta.
`0x06`	ERRO_PERIFERICO	O DUT não respondeu: falta de ACK no I2C ou *timeout* no periférico.
`0x07`	OCUPADO	Há uma rotina em execução, ou um comando anterior ainda está sendo executado; durante a rotina, apenas PING, ROUTINE_STATUS e ABORT são atendidos.
`0x08`	ROTINA_INVALIDA	A rotina contém um passo desconhecido, um parâmetro fora da faixa, está truncada ou produziria mais resultados do que a área disponível. O índice do passo é informado.
`0x09`	SEM_RESULTADO	READ_RESULT recebido sem que haja uma rotina finalizada.

Fonte: elaborado pelo autor.

A.5 COMANDOS DIRETOS

Os Quadros 15 e 16 listam os comandos diretos. Um traço (–) indica payload vazio. Nos comandos com dados de tamanho variável, os dados são sempre precedidos da quantidade, e o total do payload não pode ultrapassar 240 bytes.

Quadro 15 – Comandos diretos de sistema, DAC, GPIO, PWM e ADC.

Código	Comando	Payload da requisição	Payload da resposta (ACK)
`0x01`	PING	–	versão do protocolo (`u8`), versão do *firmware* maior, menor e correção (3 × `u8`)
`0x02`	RESET_ALL	–	–
`0x10`	DAC_SET	canal (`u8`, 1–2), valor (`u16`, 0–4095)	–
`0x20`	GPIO_SET	pino (`u8`, 1–4), nível (`u8`, 0 ou 1)	–
`0x21`	GPIO_READ	–	máscara das entradas GPIO5–8 (`u8`)
`0x30`	PWM_SET	canal (`u8`, 1–3), frequência em Hz (`u32`, 1–100 000), razão cíclica em centésimos de por cento (`u16`, 0–10000)	–
`0x31`	PWM_STOP	canal (`u8`, 1–3)	–
`0x40`	ADC_READ	canal (`u8`, 0–2)	canal (`u8`), valor bruto de 12 bits (`u16`, 0–4095)

Fonte: elaborado pelo autor.

Algumas regras complementam os quadros. Uma frequência de PWM que não pode ser gerada exatamente pelos divisores do temporizador é aproximada pela mais próxima, sem NACK; o prescaler escolhido é o menor possível, o que dá a melhor resolução da razão cíclica. PWM_STOP leva a razão cíclica a zero com o temporizador em funcionamento, deixando a saída em nível baixo. Como mestre, o SPI usa a maior frequência que não ultrapassa a pedida (54 MHz divididos por 2 a 256). Os comandos de I2C, SPI e UART que dependem de configuração (escritas, leituras e transferências) recebem NACK NAO_CONFIGURADO se o periférico não tiver sido configurado, com exceção da carga e da leitura do mapa do escravo I2C, que funcionam sem I2C_CONFIG.

A.5.1 Emulação de dispositivos I2C e SPI

No papel de escravo, a jiga emula um sensor ou periférico, permitindo ao DUT, que atua como mestre, comunicar-se com ela como faria com o dispositivo real. No I2C, a jiga mantém um mapa de 256 registradores, carregado por I2C_SLAVE_LOAD e lido de volta por I2C_SLAVE_DUMP, o que permite tanto fornecer valores ao DUT quanto verificar o que ele escreveu. O mapa segue o padrão das memórias 24C02: o primeiro byte escrito pelo mestre é o ponteiro de registrador, e as leituras e escritas seguintes avançam o ponteiro, que volta de `0xFF` para `0x00`. No SPI, a jiga mantém

Quadro 16 – Comandos diretos de I2C, SPI e UART.

Código	Comando	Payload da requisição	Payload da resposta (ACK)
`0x50`	I2C_CONFIG	papel (`u8`, 0 escravo, 1 mestre), endereço de 7 bits (`u8`, `0x08`–`0x77`, usado como escravo), velocidade (`u8`, 0 para 100 kHz, 1 para 400 kHz)	–
`0x51`	I2C_SLAVE_LOAD	*offset* (`u8`), quantidade (`u8`), dados (*n* bytes)	–
`0x52`	I2C_SLAVE_DUMP	*offset* (`u8`), quantidade (`u8`, até 240)	dados (*n* bytes)
`0x53`	I2C_MASTER_XFER	endereço (`u8`), bytes a escrever *nw* (`u8`), bytes a ler *nr* (`u8`, até 240), dados (*nw* bytes)	dados lidos (*nr* bytes)
`0x54`	I2C_SLAVE_MASK	*offset* (`u8`), quantidade (`u8`), máscaras (*n* bytes; bit 1 = o DUT pode escrever)	–
`0x56`	I2C_SLAVE_ADDR2	segundo endereço da escrava (`u8`, `0x08`–`0x77`, ou 0 para nenhum)	–
`0x60`	SPI_CONFIG	papel (`u8`, 0 escravo, 1 mestre), modo (`u8`, 0–3), frequência em Hz (`u32`, usada como mestre, a partir de 210 938 Hz)	–
`0x61`	SPI_SLAVE_LOAD	quantidade (`u8`, 0–239), dados (*n* bytes)	–
`0x62`	SPI_SLAVE_READ	–	quantidade (`u8`), dados recebidos (*n* bytes)
`0x63`	SPI_MASTER_XFER	quantidade (`u8`, 1–239), dados (*n* bytes)	dados recebidos (*n* bytes)
`0x64`	SPI_SLAVE_REG	modo registradores (`u8`, 0 ou 1), complemento do endereço (`u8`, `0x00` ou `0x80`)	–
`0x70`	UART_CONFIG	taxa em *baud* (`u32`, 1200–1 000 000), formato 8N1	–
`0x71`	UART_WRITE	quantidade (`u8`, 1–239), dados (*n* bytes)	– (enviado ao fim da transmissão)
`0x72`	UART_READ	quantidade máxima (`u8`, até 239)	quantidade (`u8`), dados (*n* bytes)

Fonte: elaborado pelo autor.

um *buffer* de transmissão, carregado por SPI_SLAVE_LOAD, e um *buffer* de recepção, lido por SPI_SLAVE_READ; cada transação recomeça a transmissão do primeiro byte do *buffer*, completando com `0xFF` quando o mestre pede mais bytes do que foram carregados, e os bytes recebidos se acumulam até a leitura. Na UART, os bytes recebidos do DUT são guardados em um *buffer* de 256 bytes até o UART_READ. No papel de mestre, a jiga inicia as transações com um periférico do DUT por I2C_MASTER_XFER e SPI_MASTER_XFER, e o sinal de seleção de escravo do SPI é acionado no pino PD7.

A.5.2 Emulação de sensores

Três comandos complementam a escrava para emular sensores comerciais. I2C_SLAVE_MASK define, para cada registrador do mapa, quais bits o DUT pode escrever: os demais mantêm o valor carregado pelo computador, que continua podendo gravar qualquer bit com I2C_SLAVE_LOAD. Todas as máscaras começam em `0xFF`. I2C_SLAVE_ADDR2 define um segundo endereço de escrava, aplicado no próximo I2C_CONFIG de escrava; os dois endereços dividem o mesmo mapa. SPI_SLAVE_REG coloca a escrava SPI, já configurada por SPI_CONFIG, no modo registradores: em cada transação, o primeiro byte é o endereço de 7 bits, completado pelo segundo parâmetro (`0x80` para os registradores de `0x80` a `0xFF`), com o bit 7 em 1 para leitura e em 0 para escrita, e os bytes seguintes são lidos do mapa ou escritos nele (só nos bits graváveis), com auto-incremento. Entre o byte de endereço e o primeiro dado, o mestre precisa deixar uma pausa de alguns microssegundos (subseção 3.5.6).

A.6 COMANDOS DE ROTINA

O Quadro 17 lista os comandos que transferem, acompanham e interrompem uma rotina de teste.

Quadro 17 – Comandos de rotina.

Código	Comando	Payload da requisição	Payload da resposta (ACK)
`0x08`	RUN_ROUTINE	identificador (`u16`), tempo máximo em ms (`u32`), número de passos (`u8`), passos (subseção A.6.1)	–
`0x09`	ROUTINE_STATUS	–	estado (`u8`, Quadro 19), passos concluídos (`u8`), identificador (`u16`), código de erro (`u8`, 0 se não houver)
`0x0A`	READ_RESULT	*offset* (`u16`)	identificador (`u16`), tamanho total dos resultados (`u16`), até 236 bytes de resultados a partir do *offset*
`0x0B`	ABORT	–	–

Fonte: elaborado pelo autor.

A.6.1 Formato da Rotina

Os passos de uma rotina são escritos em sequência, sem separadores. Cada passo é formado por um código (`u8`) seguido dos seus parâmetros, e o tamanho de cada passo é deduzido do código e dos campos de quantidade. Os passos de estímulo, de configuração e de aquisição usam o mesmo código e a mesma codificação de parâmetros do comando direto correspondente, o que permite reaproveitar no

firmware as mesmas funções de decodificação. Dois passos existem apenas dentro de uma rotina, porque dependem de tempo. O Quadro 18 resume os passos admitidos.

Quadro 18 – Passos admitidos em uma rotina.

Códigos	**Passos**	**Parâmetros**	**Registro gerado**
`0x10`, `0x20`, `0x30`, `0x31`	DAC_SET, GPIO_SET, PWM_SET, PWM_STOP	iguais aos do comando direto	nenhum
`0x50`, `0x51`, `0x54`, `0x56`, `0x60`, `0x61`, `0x64`, `0x70`, `0x71`	configuração e carga de I2C, SPI e UART	iguais aos do comando direto	nenhum
`0x21`, `0x40`	GPIO_READ, ADC_READ	iguais aos do comando direto	o mesmo payload do ACK do comando direto
`0x52`, `0x53`, `0x62`, `0x63`, `0x72`	leituras e transferências de I2C, SPI e UART	iguais aos do comando direto	o mesmo payload do ACK do comando direto
`0x78`	ESPERA	duração em µs (`u32`)	nenhum
`0x79`	ESPERA_GPIO	entrada (`u8`, 5–8), nível esperado (`u8`), tempo limite em µs (`u32`)	resultado (`u8`: 0 se o nível ocorreu, 1 se o tempo esgotou) e tempo decorrido desde o início do passo, em µs (`u32`)
`0x7A`	ONDA	gerador, destino, registro, passo e período em µs, ciclos, modo, ruído, base, amplitude e de 2 a 32 pontos (subseção A.6.2)	nenhum

Fonte: elaborado pelo autor.

O passo ESPERA_GPIO aguarda até que a entrada indicada assuma o nível esperado, ou até o tempo limite. O instante registrado é o da transição detectada, e o registro traz também o tempo decorrido desde o início do passo; como o passo começa logo após o estímulo aplicado no passo anterior, esse tempo corresponde ao tempo de reação do DUT. Esgotar o tempo limite não interrompe a rotina: é um resultado, registrado como tal, a ser avaliado pelo computador.

Se a entrada já estiver no nível esperado quando o passo começa, o registro é feito na hora, com tempo decorrido zero. O tempo máximo da rotina, informado em RUN_ROUTINE, vai de 1 ms a 1 h (3 600 000 ms): a base de tempo de 1 µs tem 32 bits e dá a volta em cerca de 71 minutos, de modo que uma hora garante que nenhum instante registrado se repita dentro de uma rotina.

Antes de iniciar a execução, a jiga valida a rotina inteira: códigos, parâmetros, integridade do encadeamento dos passos e tamanho máximo dos resultados que ela pode produzir, calculado a partir dos passos de aquisição. Se algum passo for inválido, a rotina é recusada com NACK ROTINA_INVALIDA e nada é executado; desse modo,

uma rotina nunca fica executada pela metade por causa de um erro de codificação.

A.6.2 Passo ONDA

O passo ONDA começa uma onda em segundo plano e termina na hora: a rotina segue para o passo seguinte com a onda em andamento. Seus parâmetros, em ordem, são: gerador (`u8`, 0 a 5; uma onda nova substitui a do mesmo gerador ou do mesmo destino), destino (`u8`: 0 DAC, 1 razão do PWM, 2 registrador de 16 bits com sinal, 3 registrador de 16 bits sem sinal, 4 registrador de 20 bits no formato do BME280), registro (`u8`: canal do DAC ou do PWM, ou o primeiro registrador no mapa), passo em µs (`u32`), período em µs (`u32`), ciclos (`u16`; 0 para até o fim da rotina), modo (`u8`: 0 interpolação linear, 1 degrau), ruído (`u16`, amplitude uniforme em valor bruto), base (`s32`), amplitude (`s32`), número de pontos (`u8`, 2 a 32) e os pontos (`u16` cada). Os pontos são igualmente espaçados no período, e o valor bruto escrito no destino é base $+$ amplitude $\cdot$ ponto$/65535$, saturado na faixa do destino. O passo mínimo é de 100 µs, exceto no DAC, que aceita de 1 a 99 µs desde que o período não passe de 1024 passos. Ao fim dos ciclos, o destino fica no último ponto.

A.6.3 Formato dos Resultados

Cada passo de aquisição gera um registro, gravado em uma área de resultados de 2.048 bytes, composto pelo índice do passo na rotina (`u8`, a partir de 0), pelo instante da medição em microssegundos contados a partir do início da rotina (`u32`), pela quantidade de bytes de dados (`u8`) e pelos dados. A jiga não avalia os registros: o veredito é emitido pelo computador, que compara as medições com os critérios do caso de teste. Como os resultados podem ultrapassar o tamanho de um quadro, o computador os lê em partes, com comandos READ_RESULT de *offset* crescente, até completar o tamanho total informado.

A.6.4 Estados de Execução

O Quadro 19 apresenta os estados informados por ROUTINE_STATUS. Os resultados da última rotina permanecem disponíveis até o próximo RUN_ROUTINE ou RESET_ALL, que os descarta; nos estados ABORTADA, TEMPO_ESGOTADO e ERRO, eles contêm os registros dos passos executados até a interrupção.

A.6.5 Regras de Execução

Enquanto uma rotina está em execução, a jiga atende apenas PING, ROUTINE_STATUS e ABORT, além das retransmissões (Apêndice A.7); qualquer outro comando, inclusive um novo RUN_ROUTINE, recebe NACK OCUPADO. Ao receber ABORT, a jiga interrompe a rotina, inclusive durante uma espera, e responde ACK; o

Quadro 19 – Estados de execução de uma rotina.

Valor	Estado	Significado
0	OCIOSA	Nenhuma rotina desde a inicialização ou desde o último RESET_ALL.
1	EM_EXECUCAO	A rotina está sendo executada.
2	CONCLUIDA	Todos os passos foram executados.
3	ABORTADA	A execução foi interrompida por ABORT.
4	TEMPO_ESGOTADO	A rotina ultrapassou o tempo máximo informado em RUN_ROUTINE e foi interrompida.
5	ERRO	Um passo falhou durante a execução (por exemplo, ERRO_PERIFERICO); o código de erro é informado.

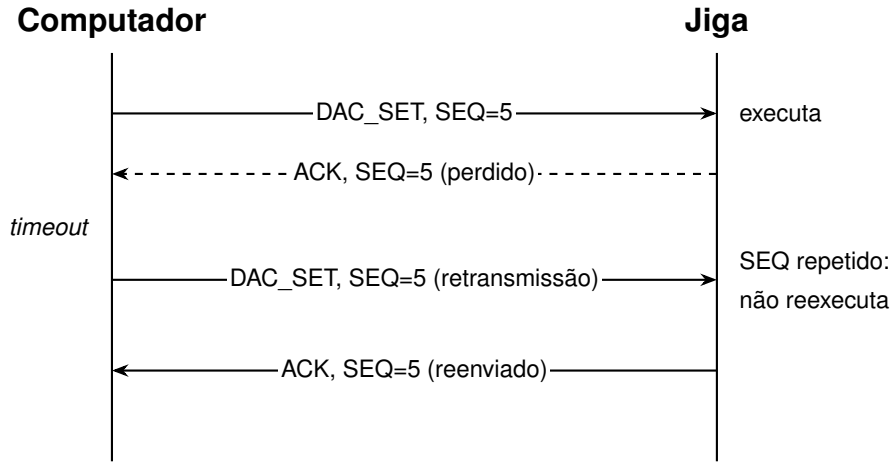
Fonte: elaborado pelo autor.

estado passa a ABORTADA. Ao término da rotina, em qualquer dos estados finais, a jiga retorna ao estado seguro (Apêndice A.8), de modo que cada rotina começa sem estímulos herdados da anterior.

A.7 SEQUENCIAMENTO E RETRANSMISSÃO

O computador espera a resposta de cada comando antes de enviar o próximo (modelo *stop-and-wait*). Se a resposta não chegar dentro do tempo limite, ou chegar com CRC inválido, o computador retransmite o comando **com o mesmo SEQ**, até um número máximo de tentativas, após o qual sinaliza falha de comunicação ao usuário. A Figura 14 ilustra o caso de perda da resposta.

Figura 14 – Retransmissão após perda da resposta.



Fonte: elaborado pelo autor.

A jiga guarda o SEQ e a resposta do último comando executado. Se receber um quadro com o mesmo SEQ, reenvia a resposta guardada **sem executar o comando novamente**. Isso torna a retransmissão segura: um comando cuja resposta se perdeu não é aplicado duas vezes (por exemplo, uma escrita I2C duplicada) e, no caso de RUN_ROUTINE, a rotina em andamento não é reiniciada. Uma retransmissão que

chega enquanto o comando original ainda está em execução é ignorada: a resposta sai uma vez, quando ele termina.

O SEQ guardado vale por **sessão**. A sessão começa quando a aplicação abre a porta serial, o que o computador sinaliza ativando a linha DTR (requisição SET_CONTROL_LINE_STATE do USB-CDC); nesse momento a jiga esquece o último SEQ e a resposta guardada. Sem essa regra, uma aplicação reaberta que começasse pelo mesmo SEQ do último comando da sessão anterior receberia a resposta antiga, sem que o comando fosse executado. A resposta OCUPADO não é guardada, pois o comando recusado não foi executado e pode ser reenviado com o mesmo SEQ depois. Quando dois comandos chegam no mesmo bloco de dados da USB, o OCUPADO do segundo pode chegar antes do ACK do primeiro; o computador associa cada resposta ao comando pelo SEQ, e não pela ordem de chegada.

Os valores de temporização são parâmetros configuráveis. Adotam-se um tempo limite de resposta de 100 ms, 3 tentativas por comando (o envio original e até duas retransmissões), um intervalo de 20 ms entre consultas ROUTINE_STATUS e um tempo limite de 20 ms entre bytes na jiga, após o qual um quadro incompleto é descartado. Os testes de desempenho confirmaram a folga desses valores: o tempo de ida e volta medido ficou abaixo de 3,2 ms em 11 000 comandos (Capítulo 4), cerca de trinta vezes menor que o tempo limite.

A.8 ESTADO SEGURO

Na inicialização, ao receber RESET_ALL e ao final de cada rotina, a jiga primeiro para as ondas em andamento e depois leva todas as saídas a um estado seguro: DAC no código 0 (cerca de 50 mV, limite inferior do *buffer* de saída), saídas GPIO1 a GPIO4 em nível baixo, PWM com razão cíclica zero (saída em nível baixo) e barramentos I2C, SPI e UART desligados, com o mapa do escravo I2C zerado, todas as máscaras em `0xFF`, sem segundo endereço, a escrava SPI fora do modo registradores, os *buffers* do SPI e da UART vazios e o pino de seleção do SPI (PD7) em nível alto. Isso impede que estímulos de uma execução anterior permaneçam aplicados ao DUT.

A.9 EXEMPLOS DE QUADROS

Os Quadros 20 e 21 apresentam exemplos reais de quadros do protocolo, com o CRC calculado segundo a Apêndice A.3, em hexadecimal.

O Quadro 21 acompanha uma rotina de três passos, com identificador 7 e tempo máximo de 1 s: aplicar 2,0 V no DAC1 (código 2482), aguardar até 500 ms que a entrada GPIO5 vá a nível alto e ler o canal A0. No exemplo, a transição do GPIO5 ocorreu 42,350 ms após o início da rotina, 42,338 ms após o início da espera (tempo de reação do DUT), e a leitura do ADC, 21 µs depois da transição.

Quadro 20 – Exemplos de quadros de comandos diretos (bytes em hexadecimal).

Mensagem	**Quadro**
PING, SEQ=1	`7E 00 01 01 8C EF`
ACK do PING (protocolo 1, *firmware* 0.1.0)	`7E 04 01 81 01 00 01 00 AC B8`
DAC_SET, SEQ=2, canal 1, valor 2048	`7E 03 02 10 01 00 08 EC 29`
ACK do DAC_SET, SEQ=2	`7E 00 02 90 47 29`
NACK, SEQ=2, comando `0x10`, erro `0x02`	`7E 02 02 FF 10 02 B5 54`
ADC_READ, SEQ=3, canal 0	`7E 01 03 40 00 E8 A6`
ACK do ADC_READ, SEQ=3, canal 0, valor 2047	`7E 03 03 C0 00 FF 07 9E EE`
PWM_SET, SEQ=4, canal 1, 1 kHz, 50%	`7E 07 04 30 01 E8 03 00 00 88 13 5A 47`

Fonte: elaborado pelo autor.

Quadro 21 – Exemplos de quadros de uma rotina (bytes em hexadecimal).

Mensagem	**Quadro**
RUN_ROUTINE, SEQ=5	`7E 14 05 08 07 00 E8 03 00 00 03 10 01 B2 09 79 05 01 20 A1 07 00 40 00 FB CD`
ACK do RUN_ROUTINE, SEQ=5	`7E 00 05 88 E9 23`
ROUTINE_STATUS, SEQ=6	`7E 00 06 09 13 F7`
ACK do ROUTINE_STATUS, SEQ=6 (em execução, 1 passo concluído)	`7E 05 06 89 01 01 07 00 00 12 1D`
ACK do ROUTINE_STATUS, SEQ=7 (concluída, 3 passos)	`7E 05 07 89 02 03 07 00 00 C9 A6`
READ_RESULT, SEQ=8, *offset* 0	`7E 02 08 0A 00 00 8D 17`
ACK do READ_RESULT, SEQ=8 (2 registros, 20 bytes)	`7E 18 08 8A 07 00 14 00 01 6E A5 00 00 05 00 62 A5 00 00 02 83 A5 00 00 03 00 FF 07 51 36`
ABORT, SEQ=9	`7E 00 09 0B 6F C7`
NACK, SEQ=9, comando `0x08`, erro OCUPADO	`7E 02 09 FF 08 07 D5 90`

Fonte: elaborado pelo autor.